Part IX

AI, VLMs and Semantic Navigation SLAM

Simultaneous Localization and Mapping (SLAM)

How to Build a Vision-Based Navigation and Localization Drone



Editor: Lee Shihyuan (李世淵)

LINE OA @prof

Taiwan-Authored University Textbook

@prof

SLAM

Simultaneous Localization and Mapping, SLAM

Part IX AI, VLMs and Semantic Navigation SLAM

Chapter 32

Semantic SLAM: Giving the Map a Meaning

Editor: Lee Shihyuan (李世淵) LINE OA @prof

Assistant: Aaron Lee, Anne Lee

University Intelligent Robotics Program / High School AI and Robotics Club Course

Learning Objectives

Thirty-one chapters on, students can now have a robot answer where am I and what does the map look like. This chapter adds the third question: what is this place? We begin with the limits of classical SLAM, explain the division of labor among AI vision models within SLAM and the four ways of representing a semantic map, then take up two practical difficulties—dynamic objects and the semantic cost map—and close with a survey of the international research landscape. By the end of this chapter students should be able to:

1. Explain the limits of a geometric map, and why the three levels of robot intelligence are laid one upon another rather than replacing one another.
2. Distinguish the roles of five classes of AI vision model in SLAM: detection, segmentation, depth, description, and scene understanding.
3. Explain the four levels of a semantic map, and why open vocabulary and scene graphs have become the focus of recent research.
4. Explain why a dynamic object pollutes a map, and design a multi-pass consistency rule for removing it.
5. Explain how semantics enter the cost map and change a navigation decision, and estimate what that costs and what it buys.
6. Explain how an error in a semantic label propagates onto the map through the extrinsics and the localization.
7. Give examples of the principal lines of research in semantic SLAM internationally, and of the systems that represent them.

Thirty-one chapters on, students can now have a robot answer where am I and what

does the map look like. But that is not enough—a robot of real practical use must also answer what have I seen, what does this place stand for, and what should I do next. This chapter is the first of Part IX, and the doorway into the decade to come. By the end of it students should be able to:

1. Explain the limits of classical SLAM, and why AI has become a necessary supplement to it.
2. Explain the division of labor among YOLO, SAM, Depth Anything, DINO and CLIP in robot perception.
3. Explain the line of development from Visual SLAM to Semantic SLAM and on to embodied AI navigation.
4. Plan the collection and labeling of training data for a robot, and train a model that recognizes scenes about a campus.
5. Explain how a VLM understands a navigation instruction given in natural language, and how RAG supplies spatial knowledge of a campus.
6. Explain the key techniques of deploying AI at the edge: quantization, distillation, and the management of memory.
7. Analyze the risks of AI-assisted SLAM, and look ahead to the union of SLAM with foundation models.

Core Concepts

Term	Meaning	Related Sections
Semantic map	Category, instance and relational information attached to a geometric map	32.3

Instance segmentation	Cutting an image into separate objects rather than categories alone	32.2
Open vocabulary	Categories need not be defined in a fixed list beforehand	32.3, 32.6
Three-dimensional scene graph	Describing space by an object–region–floor hierarchy and its relations	32.3
Removal of dynamic objects	Keeping what moves out of the map	32.4
Semantic cost map	Turning what a thing is into a cost for navigation	32.5
Propagated labeling error	The semantics right but the extrinsics or the localization out, so the label still goes on crooked	32.5, Chapter 10
Geometry–semantics–task	The three levels of robot intelligence, each holding up the one above	32.1

中文名詞	English Term	Explanation
語意 SLAM	Semantic SLAM	SLAM that annotates a geometric map with objects and the meaning of a scene.
具身智慧	Embodied AI	Artificial intelligence that learns and acts by interacting with a physical environment.
開放詞彙	Open-Vocabulary	Not bound by predefined categories, able to recognize objects unseen at training time.
物件目標導航	ObjectNav	A navigation task that takes an instruction

中文名詞	English Term	Explanation
		to go and find a certain object.
場景圖	Scene Graph	A structure describing the objects of a scene and their spatial relations as nodes and relations.
視覺語言模型	VLM	A multimodal model that understands images and language at once.
視覺語言動作模型	VLA	A model that produces action commands directly from images and language.
檢索增強生成	RAG	Retrieving relevant material before generating an answer, so as to improve its correctness.
模型蒸餾	Distillation	A method of training that transfers the ability of a large model to a small one.
量化	Quantization	Representing a model's parameters in fewer bits to lower memory and computation.
邊緣部署	Edge Deployment	Running AI inference on the computing platform aboard the robot itself.
幻覺	Hallucination	The phenomenon of AI producing content that looks reasonable and is in fact wrong.
世界模型	World Model	A model that learns the dynamics of an environment and can predict its future state.

32.1 Why Classical SLAM Still Needs AI

What the first thirty chapters of this book built is an extremely solid geometric system. It can tell you precisely where the robot is and what shape the environment has—but it has one fundamental blind spot.

Classical SLAM Knows Shape and Not Meaning

A robot running Cartographer, facing a door, knows this: 1.2 meters ahead there is a gap 0.9 meters wide, with a continuous plane on either side. It does not know that the thing is a door—does not know that a door can be passed through, that a door may close, that behind a door there is usually another room.

The difference is of no consequence in a simple task (avoiding an obstacle does not require knowing what the obstacle is), and yet it is fatal in a real service task:

- The user says go to the registrar's office, and the map holds only coordinates, no names.
- There is a puddle on the floor: geometrically it is flat ground, semantically the robot ought to go around it.
- Ahead is a glass door, which the LiDAR cannot see (Chapter 8) but which vision can recognize by its frame and handle.
- A person is standing in the corridor: geometrically an obstacle, semantically someone the robot should slow down and give way to, rather than skirt as though he were a wall.

The Gap Between the Three Levels

Level	The question it answers	Technique	Chapters of this book
Geometric	Where am I? What shape is the environment?	SLAM	Chapters 1–30
Semantic	What have I seen? What sort of place is this?	Object detection, segmentation, VLMs	This chapter
Task	What should I do next?	LLM agents, VLAs	This chapter

Table 32-1 The Three Levels of Robot Intelligence

These three levels are laid one upon another and do not replace one another—the most important idea in this chapter. Without a solid geometric level beneath it, the semantic level leaves the robot knowing that the thing is a door and unable to reach the doorway; and without a semantic level above it, the geometric level leaves the robot able only to execute coordinate commands, and unable to interact naturally with people.

What AI Brings

1. An enlargement of perception: recognizing objects, segmenting a scene, estimating depth—turning pixels into information that means something.
2. An interface of language: letting the user give a task in natural language rather than by entering coordinates.
3. Inference from common sense: a washbasin is usually in a toilet or a tea room—knowledge of this kind cannot be derived from a geometric map, and yet it raises the efficiency of exploration a great deal (Section 24.5).
4. The ability to generalize: to judge sensibly even when faced with an environment or an object not seen during training.

32.2 The Role of AI Vision Models in SLAM

Sections 23.4 and 24.5 introduced YOLO and VLMs; this section sets out systematically the division of labor among the classes of vision model—a map for planning an AI perception system.

The Division of Labor Among Five Classes of Model

Recent surveys point out the strengths and weaknesses of each family with some clarity. Image-text contrastive models (CLIP, SigLIP) are the default front end of a vision-language model, good at semantic retrieval and weaker at dense prediction; self-supervised vision transformers (DINOv2, DINOv3) retain fine spatial structure because of their training objective and do excellently at dense prediction—segmentation, depth, correspondence—though their features are not aligned with language; segmentation foundation models (SAM, SAM 2) are the experts at proposing and tracking objects; and the dense prediction specialists (Depth Anything, MiDaS) handle depth.

Class of model	Representatives	What it is good at	Its role in SLAM
Object detection	YOLO	Boxing an object's category in real time	Semantic labeling, removal of dynamic objects
Image-text contrastive	CLIP, SigLIP	Open-vocabulary semantic retrieval	Place recognition, queries in language
Self-supervised	DINOv2 / v3	Dense prediction,	Feature extraction, segmentation

Class of model	Representatives	What it is good at	Its role in SLAM
vision		spatial structure	
Segmentation foundation models	SAM, SAM 2	Proposing and tracking objects	Precise object boundaries, tracking what moves
Depth estimation	Depth Anything	Estimating depth from a single image	Supplying depth for a monocular camera, dense reconstruction

Table 32-2 The Roles of Five Classes of AI Vision Model in SLAM

One Crucial Engineering Constraint

Here is a fact of great importance to deploying a robot: a practical video system on a wearable device, a robot, or a smart camera cannot possibly carry one backbone network for every capability; it can neither compromise on capability nor bear the cost in memory and latency.

Put otherwise: although every task has its own best specialized model, a robot's computing resources will not allow five large models to run at once. In practice a trade-off must be made—choose one or two critical models, or use a unified backbone that can serve several tasks. This is precisely the thinking of Chapter 25, hardware and algorithm designed together, carried into the age of AI.

Advice to the Reader of This Book

- To begin: start from YOLO. It is mature, light, and rich in resources, and produces a visible result at once (the implementation of Section 32.4).

- Going further: add a depth estimation model to supply what a monocular camera lacks (Section 9.4), or use a segmentation model to obtain precise object boundaries.
- For research: explore open-vocabulary models, so that the robot can recognize objects not defined at training time—the key to whether semantic navigation can be made practical.

32.3 From Visual SLAM to Semantic SLAM to Embodied AI Navigation

This section sets out that line of development, so that students may see clearly where what they have learned stands and in what direction it runs.

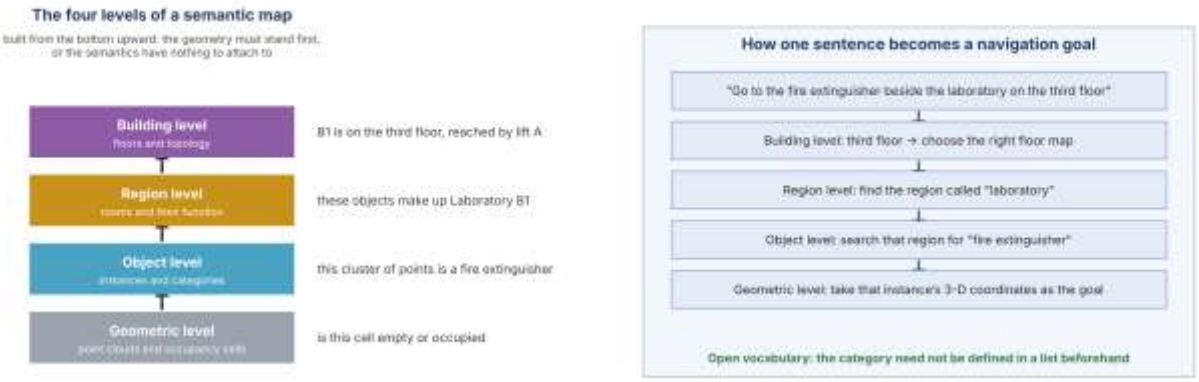
Three Stages

5. Visual SLAM (Chapters 20 to 23): the output is the camera's trajectory and a sparse or dense geometric map. Every point in the map has coordinates and no meaning.
6. Semantic SLAM: object recognition and segmentation are carried out while the map is built, and semantic labels are attached to the elements of the map. A point in the map is no longer merely there is something here but this is part of a chair.
7. Embodied AI navigation: the robot has not only a semantic map but the ability to understand a task, reason from common sense, and plan an action. It can accept an instruction such as go and turn off the light in the meeting room and carry it out by itself.

Two Ways of Doing Semantic SLAM

- As post-processing: build the geometric map first with classical SLAM, then project the results of object detection onto it. Simple to implement and loosely coupled in its modules, it is the first choice for teaching and for a student project (the route taken by the implementation of Section 32.4).
- Tightly coupled: bring the semantic information into SLAM's own estimation—replacing point-level association with object-level association, say, or using semantic priors such as a wall is a plane and a floor is horizontal to constrain the optimization. More accurate, and more complicated to implement.

Figure 32-1 The Four Levels of a Semantic Map, and How One Sentence Becomes a Navigation Goal



A semantic map is often misunderstood as sticking labels on a map, but semantics that are really useful have a hierarchy. The left half of the figure divides it into four levels. At the bottom is the geometric level, which answers only whether this cell is empty or occupied; above it the object level, which takes a cluster of points to be one instance—this is a fire extinguisher; above that the region level, which assigns a group of objects to one functional space—Laboratory B1; and at the top the building level, which records floors and the topology among them—B1 is on the third floor and is reached by lift A.

The hierarchy is built from the bottom upward and the order cannot be reversed:

without correct geometry the object instances fall in the wrong places, and without objects there is nothing from which to induce a region. This too is why this book puts semantics in Part IX—it needs all eight preceding parts in place before it can stand at all.

The right half of the figure then demonstrates what the hierarchy is for: parsing a sentence from the top downward. The user says go to the fire extinguisher beside the laboratory on the third floor; the system first chooses the right floor map at the building level, then finds the laboratory at the region level, then searches the object level within that region for a fire extinguisher, and finally takes that instance's three-dimensional coordinates as the navigation goal. Each level need answer only one simple question, rather than one model having to handle the whole sentence at once.

Here is exactly where the value of a three-dimensional scene graph lies. If the map were no more than a heap of cells marked with categories, the spatial relation beside the laboratory could not be expressed at all; and an open vocabulary frees the last step of the search from a fixed list of categories—even with no category fire extinguisher at training time, it can still be found by matching text features.

What the Four Levels Mean in Practice

The geometric level decides safety: whether a thing will be run into depends on this level alone. A semantic error is not fatal; a geometric one is.

The object level decides interaction: whether it can go and fetch that thing depends on whether the instance is correctly segmented and located.

The region level decides efficiency: only with the notion of a room can one say clean the kitchen alone or do the third floor first (Chapters 16 and 19).

The building level decides reachability: navigating between floors needs topology, and a lift is a typical topological node rather than a geometric obstacle.

Advice for a project: reaching the object level alone is already of value; the region level can be segmented automatically by the distance transform method of Section 16.4; and the building level can usually be labeled by hand.

The Frontier: Open Vocabulary and Scene Graphs

Recent research is pressing forward in two directions:

- Open-vocabulary online semantic mapping: freeing a SLAM system from a predefined set of categories, so that it can recognize and label arbitrary objects while it maps—which matters greatly given the complexity and variety of real environments.
- Scene graph representations: recording not merely that there is a chair here but the structure of relations—the chair is beside the table, the table is in the meeting room. Recent work has shown zero-shot embodied navigation supported by a multimodal three-dimensional scene graph.

The value of a scene graph lies in its ability to support inference: when the user says go and fetch the cup on the table, the robot must understand the spatial relation on the table—which a purely geometric map cannot express.

The Convergence of the Academy and Industry

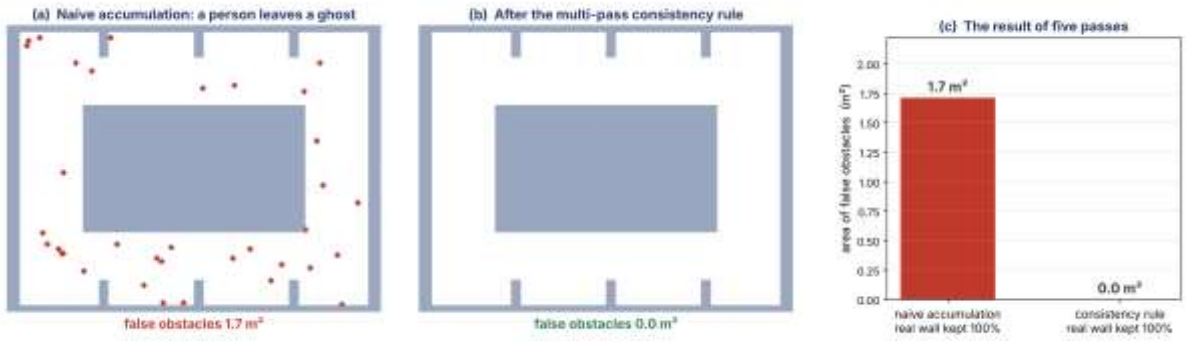
It is worth noting that along this line of development the academy and industry are converging quickly. Research in the field has already brought SLAM together with semantics, three-dimensional Gaussians, the bird's-eye view, navigation, large

language models, and multi-stage operation into a single range of concerns for one research community. For students this means that the geometric foundation this book teaches is precisely the ticket into that lively field.

32.4 A Dynamic Environment: What Moves Should Not Go into the Map

The most direct use of semantics, and the earliest the industry adopted, is in fact not recognizing what a thing is but recognizing what ought not to be recorded. Section 13.5 discussed how upper and lower bounds on the occupancy probability keep a map malleable, but that is a passive forgetting; with semantics the robot can judge actively: this is a person, so do not write it into the map.

Figure 32-2 The Ghosts People Leave Behind, and How to Clear Them Away



In this experiment the robot goes five times around the same floor, with seven people at random positions in the environment on every pass. Panel (a) is the result of naive accumulation: a cell is recorded as an obstacle if it was ever observed occupied on any pass. Grey is the real wall and red the false obstacles added to the map—1.7 square meters in all. They are scattered up and down the corridor, and every patch is a place where somebody happened to be standing on some pass.

Panel (b) is the same with a multi-pass consistency rule added: a cell must be observed

occupied on at least three of the five passes before it is recorded as an obstacle. The result is that the false obstacles fall from 1.7 square meters to zero, while the real wall is still kept at 100 percent. The reason is plain—the wall is there on every pass, a person is not.

The rule works because it exploits one semantic fact: permanent structure and temporary objects behave entirely differently in time. It is worth noting that no deep learning model is used here at all, only the statistical method of voting across passes—which is the concrete implementation of the layered map strategy of Section 19.6.

When, then, is AI really needed? The answer is when you cannot wait for five passes. Voting across passes takes time, and the robot must avoid the person walking toward it on the very first pass. What is needed then is instance segmentation in real time: recognize in a single frame that this is a person, and exclude the corresponding points from the mapping outright. The two methods are used together in practice—AI handles the present moment, voting handles the long term.

The Three Lines of Defense Against Dynamic Objects

The immediate layer (milliseconds): remove known dynamic categories—people, trolleys—within a single frame by instance segmentation or object detection. Quick to react, and recognizes only the categories it was trained on.

The short-term layer (seconds to minutes): detect a moving object by the inconsistency between consecutive frames, needing no knowledge of categories, though insensitive to slowly moving objects.

The long-term layer (between passes): the multi-pass consistency voting of this figure. It needs no model at all and handles changes of every kind, at the cost

of needing time to accumulate.

The three lines of defense solve problems at different scales of time, and a practical system usually uses all three rather than picking the best one.

32.5 How Semantics Change a Navigation Decision

This section explains how semantics really change the behavior of the planner, and what the traps are.

From Category to Cost: Three Ways of Injecting It

- Injection at the region level: mark a whole region with a category (a forbidden zone, a busy thoroughfare), then map the category to a cost. The simplest, and the one most often used for forbidden zones set by hand.
- Injection at the object level: when a particular object is detected (glass, a mirror, the head of a staircase), add a ring of cost around it. Suited to objects on which a sensor will fail.
- Learned traversability: predict directly from the image how easy this place is to drive over, with no explicit category in between. Popular in recent years outdoors and on unstructured terrain, at the cost of making a failure hard to explain.

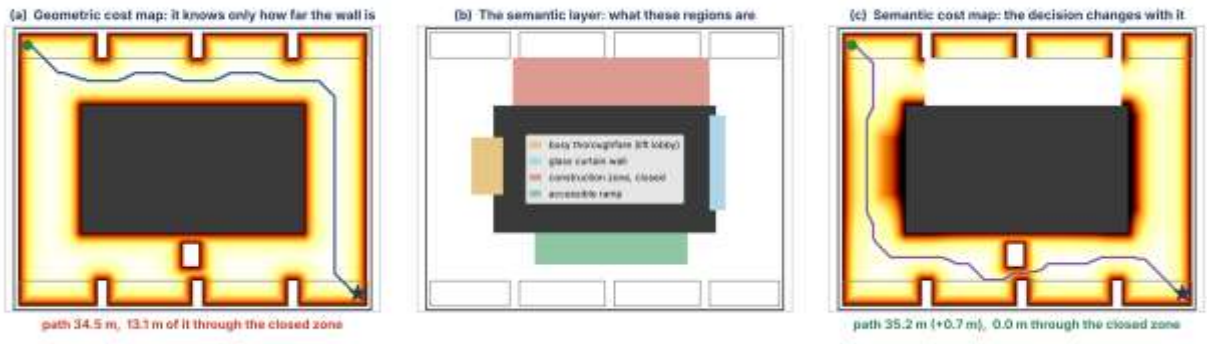
A Trap Easily Overlooked: the Label Goes on Crooked

The accuracy of semantic segmentation is often treated as the only measure, but for SLAM recognizing it correctly and putting it in the right place are two different things. A model that recognizes a fire extinguisher perfectly will, if the extrinsics between camera and LiDAR are two degrees out, put the label tens of centimeters from where it belongs on the map.

Formula Two has already written out that chain of propagation: a detection on the image must be back-projected through the camera intrinsics, then carried through the extrinsics from camera to chassis, from chassis to odometry, and from odometry to map, before it lands at a final map coordinate. An error at any link of that chain accumulates. Put otherwise, the quality of a semantic map depends at once on the AI's rate of recognition and on the quality of the coordinate transforms of Chapter 10—and in practice the latter is more often the bottleneck.

This leads to a very practical order of debugging: when a semantic label goes on crooked, do not rush to change to a larger model. First verify the extrinsics and the localization with an object whose position is known (a target stuck on the wall), and only once the geometric chain is confirmed sound go back and examine the recognition model. The contents of Parts I to VIII of this book are still the foundation here.

Figure 32-3 The Same Map, and How the Decision Changes Once Semantics Are Added



The experiment of this book uses a floor whose corridors run in a loop: at the center a core of lifts and machine rooms, corridors around it, and classrooms and laboratories outside them. The robot is to carry something from the room at the top left to the one at the bottom right—and since either side of the core will serve, which side to take

becomes a real decision.

Panel (a) is the cost map with geometric information alone. The redder the color the closer to an obstacle and the higher the cost, so the planner tends to keep to the middle of the corridor. It chose the clockwise route along the top, 34.5 meters in all—and by a purely geometric standard that is the right answer. The trouble is that 13.1 meters of that route pass through a construction zone that is closed, and the geometric map has no idea the place is closed: to it that is merely a stretch of corridor with nothing in the way.

Panel (b) is the same floor with a semantic layer added. Four regions are marked out: the busy thoroughfare by the lift lobby, the glass curtain wall, the construction zone, and the accessible ramp. None of this can be seen geometrically at all—the glass curtain wall is a thing the LiDAR cannot even see, and it appears on the map only because another sensing modality and a semantic model put it there.

Panel (c) is the result of turning the semantics into cost. The closed zone is set impassable, the thoroughfare and the glass wall are given an extra cost that decays with distance, and the ramp is given a lower cost because it is easy going. The planner therefore takes the counterclockwise route along the bottom, 35.2 meters in all—only 0.7 meters more, and yet the 13 meters of passage through the closed zone fall to nothing. Those 0.7 meters are the price of semantics, and what they buy is a route that is compliant, safe, and better matched to what a person would expect.

How a Semantic Cost Should Reasonably Be Set

Hard constraints against soft costs: a forbidden zone or a construction zone should be set impassable (a hard constraint); a busy thoroughfare or a glass

curtain wall suits an extra cost that can be weighed against others (a soft cost).

The decay distance matters more than the number: this figure sets a decay distance of 1.0 meters for the glass curtain wall and 1.3 for the busy thoroughfare. That distance decides how far away the robot begins to keep clear, and affects its behavior more than the numerical cost does.

A negative cost is useful too: the accessible ramp is given a low cost, which amounts to encouraging the robot to use it. Semantics are not only for avoiding bad places but for preferring good ones.

Beware of over-weighting: set every semantic region to a high cost and the planner becomes conservative, or finds no path at all. In practice it is advised to add one or two semantic layers at a time, and to confirm the behavior is reasonable before adding more.

32.6 A Map of the International Research: Three Lines in Semantic SLAM

Semantic SLAM has developed internationally along roughly three lines, each with its representative research groups and systems. This section offers a map, so that students know where to look for material.

One: Metric-Semantic SLAM—Fitting Semantics into an Existing SLAM Framework

The claim of this line is: do not start afresh, but add a layer of semantics to a mature SLAM system. Representative work attaches instance segmentation to the back end of a visual-inertial SLAM, maintaining object-level instances while the geometric map is built and organizing them in a three-dimensional scene graph. Its merit is that it inherits

the stability of the existing system, and it fits the architecture of Parts I to VIII of this book best.

Two: Open-Vocabulary Maps—Escaping the Fixed List of Categories

Classical semantic segmentation can recognize only the few dozen categories defined at training time, and the things in a real site are far more than that. The open-vocabulary line stores the features of a vision-language model directly in every element of the map and matches text against them at query time—so that a query never defined in advance, find the red fire extinguisher, will work. The price is a sharp rise in memory use, and a result that cannot be guaranteed.

Three: Neural Fields and Renderable Representations—Localization and Reconstruction in One

Three-dimensional Gaussian splatting, mentioned in Sections 17.8 and 20.8, has its counterpart in the semantic field: store semantic features in every Gaussian element as well, so that the map can both render a lifelike image and answer a semantic query. This line has the advantage in the quality of its reconstruction and the highest cost in computation and memory, and is at present used more for offline reconstruction than for navigation in real time.

The trade-off among the three lines may be understood thus: the first is the steadiest and the best suited to deployment; the second the most flexible and the best suited to an open site; the third the handsomest and the best suited to demonstration and offline analysis. For a student project the first is advised—it is wholly compatible with this book's stack of techniques, and when something goes wrong you know which layer to look at.

How to Follow the Progress of This Field

Conferences: ICRA, IROS, RSS, CVPR and CoRL are the chief venues at which semantic SLAM is published.

Keywords: semantic SLAM, open-vocabulary mapping, 3D scene graph, instance-aware mapping, dynamic SLAM.

Datasets for assessment: indoors, chiefly ScanNet and Replica; outdoors and for autonomous driving, SemanticKITTI and nuScenes.

Where to start implementing: attach an off-the-shelf segmentation model to an existing ORB-SLAM or RTAB-Map, project the result onto the map, and walk the chain of propagation of Formula Two with your own hands—this gives a better feel for it than reading ten papers.

Formulas and Procedures

One: the Measures by Which Object Detection Is Assessed

In training a detection model such as YOLO the measure most used is the mean average precision:

$$mAP = (1/C)\ \Sigma_c AP_c$$

where C is the number of categories and AP_c the average precision of category c (the area under the precision-recall curve). Common variants are mAP@0.5, computed at an intersection-over-union threshold of 0.5, and mAP@0.5:0.95, averaged over several thresholds (the stricter of the two).

For a robot application two practical measures deserve attention as well:

- The miss rate: the proportion not detected. For objects bearing on safety (a person, the head of a staircase) a miss is far more dangerous than a false alarm.
- The performance of each category: a high overall mAP does not mean that every category is good. If the precision of the category staircase is particularly low, and that is precisely what safety turns on, then the overall figure is misleading.

Two: Projecting a Detection onto a Three-Dimensional Map

The central computation of semantic labeling is turning a detection box on the image into a three-dimensional coordinate. Taking the center pixel (u, v) of the box and its depth d, back-project by Formula One of Chapter 30:

$$P_{camera} = \left((u - c_x) \cdot Z / f_x ,\ (v - c_y) \cdot Z / f_y , Z \right) \quad , Z = d / scale$$

and then transform it into the map frame by the chain of coordinate transforms of Chapter 10:

$$P_{map} = T_{map \leftarrow odom} \cdot T_{odom \leftarrow base} \cdot T_{base \leftarrow camera} \cdot P_{camera}$$

This formula joins the AI of this chapter to the coordinate transforms of Chapter 10—semantic labeling looks like the work of AI, and yet whether it lands correctly on the map depends on the quality of the extrinsics and the localization. Which bears out once more the point of Section 32.1: the semantic level must be built upon a solid geometric level.

Four: the Complete Data Flow of AI-Assisted SLAM

Figure 32-4 The End-to-End Flow from Sensing to a Semantic Decision

① Sensors: camera, LiDAR, IMU

↓

② Geometric SLAM: localization and mapping (Chapters 13–22) → pose + geometric map

↓

③ AI perception: YOLO detection / segmentation / depth estimation (Section 32.2)

↓ project the detection into map coordinates by Formula Two

④ Semantic map: geometry + object labels + spatial relations

↓

⑤ RAG knowledge base: campus topology, the attributes of places, the rules of passage (Section 32.5)

↓

⑥ The VLM / LLM agent: understand the instruction, reason, choose the goal (Section 32.6)

↓ output a structured geometric goal

⑦ Path planning and control (Chapter 29) → velocity commands

↓

⑧ Execute → the environment changes → return to ①

【The guard of safety】step ⑦ has the right to overrule the decision of ⑥

Cases

Cases

Case 32-1 The Semantics Right, the Position Wrong

A failure extremely common in student projects: the student attaches an object detection model better than ninety percent accurate, the robot does indeed recognize the fire extinguisher in the corridor, and yet a look at the map shows the label floating a meter outside the wall, or even landing in the room next door.

The instinctive reaction is usually that the model is not good enough, so a larger model is fetched and more data labeled afresh. But the real cause is almost never in the model. What detection gives is a box on an image; to turn it into a point on the map it must pass through depth estimation, the camera intrinsics, the extrinsics from camera to chassis, and the localization of the moment—four links. The one that most often goes wrong is the extrinsics: the camera was not calibrated again after it was mounted, or the design value was used rather than the measured one, and two or three degrees are enough to cause an offset of tens of centimeters.

The way to diagnose it is simple: take an object whose position is known (a target stuck on the wall) as the subject of the test, have the robot detect it ten times from different angles, and see whether the labeled points converge on one position. If the labeled points spread out with the viewing angle, the trouble is in the extrinsics or the localization; if they are tight but displaced as a body, the trouble is in the translation of the extrinsics; only when the labeled points themselves jump about is it necessary to go back and examine the model.

The lesson of this case is the same as that of the repeatability of localization in Section

19.5: for the output of AI to land in the right place depends on the coordinate transforms of Chapter 10 and the localization of Chapter 11. However strong the semantic layer, it stands on the shoulders of the geometric one.

Case 32-2 Why a Robot Vacuum Does Not Record Your Feet as a Wall

An application of semantics that happens every day and is seldom noticed. While a robot vacuum works, people walk about the house; if it recorded every foot it detected as an obstacle, the map would soon be strewn with walls that do not exist, and the coverage planning of Chapter 16 would fail along with it.

What it does is exactly the three lines of defense of Figure 32-2. The immediate layer: judge by infrared or by camera that what is ahead is a moving object, and go around it for the moment without writing it into the map. The short-term layer: a position that is occupied on some scans and not on others over several consecutive scans is judged an unstable observation. The long-term layer: an obstacle that has not appeared again over several cleanings has its occupancy probability lowered step by step until it vanishes from the map (the mechanism of upper and lower bounds of Section 13.5).

It is worth noting that a low-end model often has only the long-term layer, which is why you will see it record a backpack left on the floor as an obstacle and go on driving around a thing that is no longer there for days. That difference is the concrete value of semantic ability in a product—it shows itself not necessarily as being cleverer but as doing fewer stupid things.

Case 32-3 What an Open Vocabulary Costs

An open-vocabulary map sounds nearly all-powerful: no need to define categories beforehand, and the user describes in words whatever he wishes to find. But its costs

are quite concrete, and worth students knowing in advance.

The first is memory. A classical semantic map stores one category number for each element (a few bytes); an open-vocabulary map must store a feature vector from a vision-language model—commonly several hundred floating-point dimensions, which is to say a thousand bytes and more per element. For the same map the memory used may differ by two or three orders of magnitude, which is precisely the difficulty of edge deployment that Chapter 33 takes up.

The second is reliability. A model with fixed categories has at least clear measures of assessment and clear boundaries of failure; an open-vocabulary query may give wholly different results for two similar descriptions, and there is no good way of knowing in advance which queries will fail. For a navigation task that needs a guarantee of safety, this is a serious problem.

The practical course at present is a mixture: categories critical to safety (people, staircases, forbidden zones) still use a model with fixed categories, with explicit assessment and guarantees; the open vocabulary is used for interacting with the user and for queries—occasions where being wrong does nothing worse. Students may apply this principle of the division of labor to a project directly.

Implementation Activities

Implementation Activities

Implementation 32-1 A Semantic Cost Map and the Removal of Dynamic Objects

Purpose: to fit semantic information into the map and the planner with one's own

hands, to verify how it changes a decision, and to implement the multi-pass consistency removal of dynamic objects. The implementation is chiefly in Python; parts one and two need no robot hardware.

Part One: the Semantic Cost Map (for everyone)

1. Take an occupancy grid map (the product of the implementation of Chapter 13 will serve), produce a geometric cost map from it by a distance transform, plan a path from a start to a goal with A*, and record its length.
2. Mark two or three semantic regions on the map as rectangles (a forbidden zone, a busy thoroughfare, a preferred route), and assign each class a cost and a decay distance.
3. Add the semantic cost on top of the geometric cost map, plan again, and compare the lengths of the two paths and the distance each runs through a semantic region, reproducing the three numbers of Figure 32-3.
4. An experiment in sensitivity: raise the cost of the busy thoroughfare from zero by degrees, find the threshold at which the path jumps to the other route, and explain what that threshold means.

Part Two: Removing Dynamic Objects (for everyone)

1. Place several people at random in a simulated environment, run one pass of scanning and mapping, and repeat for five passes.
2. Build the map by naive accumulation (recorded as an obstacle if observed on any pass), compare it with the true map, compute the area of the false obstacles, and reproduce Figure 32-2(a).
3. Change to voting for multi-pass consistency (recorded as an obstacle only if observed on at least k passes), sweep k from 1 to 5, draw the two curves of area

of false obstacles and proportion of the real wall kept, and find the best k.

4. For discussion: if the number of passes rose from 5 to 20, how would the best k change? What does that suggest for the strategy by which an inspection robot (Chapter 19) maintains its map?

Part Three: Projecting the Semantic Label (university level)

1. With an RGB-D camera or with simulated data, run an off-the-shelf object detection model on one image and obtain the detection box and the depth.
2. Project the detection into the map frame by Formula Two, and draw the labeled point on the map.
3. Alter the rotation of the extrinsics deliberately by 1, 2 and 5 degrees, record how far the labeled point moves, and so quantify the effect of an extrinsic error, reproducing the phenomenon of Case 32-1.
4. (Going further) Detect and project the same object from five different viewpoints, and compute the spread of the labeled points as a combined measure of the quality of the whole chain of propagation.

Discussion Questions

1. How much farther does the semantic path travel than the geometric one? What does that cost buy? If the extra distance were 50 percent rather than 2, would you still choose the semantic path?
2. What is the best value of k for multi-pass consistency voting? What happens when k is too small, and when it is too large?
3. With the rotation of the extrinsics 2 degrees out, how far does a labeled point ten meters away move? How does that figure compare with the centimeter-level localization accuracy required in Section 19.5?

4. Which categories suit a model with a fixed list, and which an open vocabulary? Classify them by the consequence of being wrong.
5. (University level) If you were to design a semantic map for a campus tour robot, which four semantic layers would you put in it? How would you set the cost and the decay distance of each?

Extended Challenge (university level)

The first: implement a semantic layer editor—let the user draw regions on the map, choose a class, adjust the cost, and see at once how the planned path changes. A tool of this kind is standard equipment in inspection and service robot products. The second: raise the removal of dynamic objects from voting to a layered map: maintain two maps, a permanent layer and a transient one, localize against the permanent layer alone and avoid obstacles with both, and compare its rate of successful localization with that of plain voting (the method of assessment of Section 19.6 may be reused).

Further Reading

1. N. Yokoyama et al., "VLFM: Vision-Language Frontier Maps for Zero-Shot Semantic Navigation," ICRA, 2024. (The representative work on zero-shot semantic navigation, referred to in Chapters 24 and 32 alike.)
2. T. B. Martins, M. R. Oswald, and J. Civera, "Open-Vocabulary Online Semantic Mapping for SLAM," IEEE RA-L, 2025. (Open-vocabulary semantic mapping, the frontier direction of Section 32.3.)
3. D. Shah et al., "LM-Nav: Robotic Navigation with Large Pre-trained Models of Language, Vision, and Action," CoRL, 2022. (The pioneering work on navigation driven by a language model.)

4. Embodied AI with Foundation Models for Mobile Service Robots: A Systematic Review, arXiv, 2025. (A complete survey of embodied intelligence and foundation models, suitable as a starting point for research.)
5. Ultralytics YOLO official documentation (docs.ultralytics.com). (The training, assessment and deployment of object detection models, the chief tool of this chapter's implementation.)
6. The awesome-and-novel-works-in-slam open-source list (GitHub). (A continuously updated collection of new directions: semantic SLAM, three-dimensional Gaussians, navigation, large language models and more.)
7. NVIDIA Jetson developer documentation and the TensorRT optimization guide. (A practical reference for deploying AI at the edge, the technical foundation of Section 33.8.)

Summary

Summary

- Classical SLAM knows shape and not meaning; robot intelligence may be divided into three levels—geometric, semantic and task—which are laid one upon another and do not replace one another: without a solid geometric level beneath it, the semantic level only puts its labels on crooked.
- AI vision models divide five ways within SLAM: detection, segmentation, depth, description and scene understanding; the crucial constraint on the choice is the budget of computation and memory on an edge platform.
- A semantic map divides into four levels—geometric, object, region and building—built from the bottom upward; an open vocabulary frees the categories from having to be defined in advance, and a scene graph makes

spatial relations something that can be queried and reasoned over.

- A dynamic object written straight into the map leaves a ghost behind. The experiment of this chapter shows naive accumulation producing 1.7 square meters of false obstacles, while voting for multi-pass consistency brings that to zero and keeps 100 percent of the real wall.
- Dynamic objects are handled by three lines of defense—instance segmentation in real time, inconsistency between frames in the short term, and voting across passes in the long term—which work at different scales of time and are used together in practice rather than one instead of another.
- Semantics enter navigation through the cost map. The experiment of this chapter shows the semantic path travelling only 0.7 meters farther (34.5 → 35.2 meters) while the 13 meters of passage through a forbidden zone fall to nothing; hard constraints and soft costs should be designed separately, and the decay distance affects behavior more than the numerical cost does.
- The quality of a semantic label depends at once on the rate of recognition and on the chain of coordinate transforms; two or three degrees of error in the extrinsics are enough to cause an offset of tens of centimeters, and in debugging the geometric chain should be verified before the model is examined.
- International research runs along three lines: metric-semantic SLAM is the steadiest, the open vocabulary the most flexible, and neural fields and renderable representations give the best reconstruction at the highest cost.

Carrying Forward

This chapter has carried the map from shape to meaning. Figure 32-1 set out the four levels of a semantic map, Figure 32-2 showed what ought not to go into a map, and

Figure 32-3 demonstrated how semantics change a decision. But all of this presupposes one thing: that a good enough AI model is already in your hands.

Where does the model come from? Real data are expensive and labeling takes time, and the cost of a robot's trial and error in the real world is higher still. In the next chapter we enter simulation: Chapter 33 introduces how to train a robot on a large scale in a simulated environment and then deploy the learned policy on real edge hardware—a route of simulate first, then deploy, which is by now the standard procedure from national projects to volume production in industry.

(The chapter ends here; the next enters training in simulation and deployment at the edge.)

Taiwan-Authored University Textbook

@prof

SLAM

Simultaneous Localization and Mapping, SLAM

Chapter 33

Training in Simulation and Sim-to-Real

Editor: Lee Shihyuan (李世淵) LINE OA @prof

Assistant: Aaron Lee, Anne Lee

University Intelligent Robotics Program / High School AI and Robotics Club Course

Where This Chapter Sits in the Book

Chapter 32 gave the map a meaning—the robot knows that the thing is a door. But what after it knows? To have the robot actually walk over, push the door open, and stand firm on uneven ground, there must be a control policy behind it, and such a policy can hardly be learned from nothing on a real robot. The reason is simple: reinforcement learning needs tens of millions of attempts, and a real robot can neither afford that many falls nor spare that much time.

What this chapter takes up is exactly that contradiction: practice in simulation first, then carry what has been learned over to the real robot. The route is called sim-to-real, and since 2020 it has become all but the standard practice for legged robots and dexterous manipulation. This chapter follows NVIDIA's Isaac Sim and Isaac Lab ecosystem as its main line, because it is at present the most complete set of tools and the easiest to obtain; but the ideas this chapter is about—why parallel simulation works, where the gap between simulation and reality comes from, and how to close it—do not change with the tool.

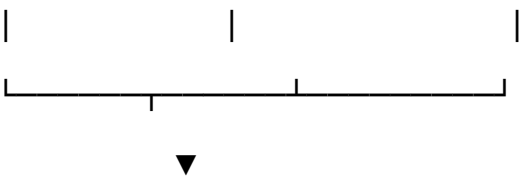
The relation of this chapter to the earlier parts is direct: the coordinates and probability of Chapters 10 and 11 are the language of a simulator, the sensor models of Chapters 8 and 9 decide what the simulator must imitate, and the edge computing of Chapter 25 is the hardware foundation of Section 33.8, on how to deploy what has been trained. This, one might say, is the chapter that carries the techniques of the whole book from running to learning.

Figure 33-0 The Link Map of Chapter 33

The foundations the earlier parts supply

Chapters 8–9 Sensors Chapters 10–11 Coordinates and Probability Chapter 25 Edge

Computing



【Chapter 33 Train in Simulation, Then Carry It to a Real Robot】

Parallel simulation ─ Sensor simulation ─ Synthetic data ─ Domain randomization ─ Edge deployment

▼

Chapter 34 VLMs, Language Navigation and Embodied Intelligence (the close of the book)

Learning Objectives

This chapter starts from why simulate first and takes up in turn the principle of parallel simulation, the simulation of sensors, the generation of synthetic data, the sources of and remedies for the sim-to-real gap, and finally deployment on an edge device. It has three figures computed from experiments, each answering one question: what parallelism really buys, where simulation and reality differ, and what domain randomization costs. By the end of this chapter students should be able to:

1. Explain why a reinforcement-learned robot policy must nearly always be trained in simulation first, and estimate the number of interactions needed.
2. Explain the division of labor between Isaac Sim and Isaac Lab, and the difference between GPU-native simulation and a classical simulator.
3. Explain how the number of parallel environments affects throughput, gradient noise and the stability of training all at once, and know the limit of each.
4. Explain how the sensors a robot commonly uses are imitated in simulation, and which parts are imitated well and which still fall short.

5. Explain how synthetic data are generated and a real scene reconstructed, and plan the sources of data for one's own project.
6. Point out the chief sources of the sim-to-real gap, and distinguish when system identification and when domain randomization is the remedy to use.
7. Explain what domain randomization costs: the trade-off between robustness and nominal performance.
8. Plan a complete route from training in the cloud to deployment on a Jetson, and estimate the memory needed after quantization.

Core Concepts

Term	Meaning	Related Sections
Sim-to-Real	Training a policy in simulation and then deploying it on a real robot	33.1, 33.6
Parallel simulation	Advancing thousands of environments at once on one GPU	33.3
GPU-native	The state living in GPU memory as tensors, with no round trip to the CPU	33.3
OpenUSD	An open scene format for describing scenes and robots	33.2
Domain randomization	Sampling environment parameters at random	33.6

	during training, buying robustness in deployment	
System identification	Measuring a real robot and working back to the simulation's parameters	33.6, Chapter 11
Synthetic data	Training data produced and labeled automatically by a simulator	33.5, Chapter 32
Domain gap	The loss of performance caused by a training distribution that does not match the deployment one	33.5, 33.6
Quantization	Representing a model's parameters in fewer bits to save memory	33.8
Distillation	Transferring the ability of a large model to a small one that can run at the edge	33.8

Table 33-1 The Core Terms of This Chapter

33.1 Why a Robot Must Learn in Simulation First

Let us state the scale of the problem plainly first.

One: the Problem of the Numbers

To train a walking policy for a legged robot by reinforcement learning, the typical requirement in samples is of the order of a billion (10^9) steps of interaction with the environment. Suppose the control frequency is 50 Hz: a billion steps come to 20 million

seconds, which is about 231 days of continuous running. And that is under the ideal assumption that the robot will not break, the battery needs no charging, and it picks itself up after every fall.

The real world has three further costs that simulation does not: the hardware breaks (one fall may damage a gearbox or a shell), a person must watch over it throughout (it cannot run unattended), and an experiment cannot be repeated (today's floor is not as damp as yesterday's, so two algorithms cannot be compared cleanly). These three together make reinforcement learning on a real machine, in the great majority of cases, not a question of cost but something fundamentally impracticable.

Two: What Simulation Offers

- Time can be sped up: simulation is not bound by real time, and one second can advance thousands of environments by tens of steps.
- Failure costs nothing: a fall merely resets the state, and breaks no part.
- The state is wholly known: the simulator knows the robot's true pose and velocity, which makes asymmetric training possible—the critic network may see the true state while the policy network sees only sensor data.
- Experiments can be repeated: fix the random seed and the same experiment can be run a hundred times, which is the premise of comparing algorithms (the methods of assessment of Chapter 31).
- Environments can be generated at will: terrain, lighting and the arrangement of objects can all be produced programmatically, a variety the real world cannot offer.

Three: What Simulation Does Not Offer

The limits of simulation must be stated just as honestly. A simulator solves the

equations you wrote into it, not the physics of the real world. Whatever was not written into those equations—the motor's rise in temperature, the backlash of the gears, the tug of the cabling, the change in the coefficient of friction once the foot pads have worn, those compensations the engineers added in the firmware to make it pleasant to use—the simulator knows nothing of. These unmodeled dynamics are the chief source of the sim-to-real gap, which Section 33.6 takes up one by one.

> **Why SLAM Took Up Training in Simulation Later than Control Did**
>
> Most of the SLAM algorithms of the first thirty-two chapters of this book need no training—they are derived, not learned. This is the fundamental difference between SLAM and a control policy: SLAM has an explicit probabilistic model and objective of optimization (Chapters 11 and 12), while nobody can write a closed-form solution for how to walk steadily on gravel.
>
> But that line is blurring. Deep-learned visual odometry, learned loop closure descriptors, and the semantic segmentation of Chapter 32 all need a great deal of data; and the generation of synthetic data in Section 33.5 is exactly the route by which SLAM too begins to benefit from simulation.

33.2 What Isaac Sim and Isaac Lab Are

The two names are often used interchangeably, though the division of labor between them is in fact quite clear.

One: the Division of Labor Between the Two Layers

	Isaac Sim	Isaac Lab
What it is	The simulator and the	A framework for robot

	scene editing environment	learning
What it does	Physics solution, sensor rendering, importing scenes and robots	Defining tasks, observations and rewards, the training procedure
Who uses it	Those making scenes, attaching ROS 2, or generating data	Those training policies
What it rests on	PhysX physics + RTX rendering + OpenUSD scenes	Built upon Isaac Sim
An analogy	The laboratory and its instruments	The lab manual and the marking sheet

Table 33-2 The Division of Labor Between Isaac Sim and Isaac Lab

Put more plainly: Isaac Sim is the world and Isaac Lab is the course. You may use Isaac Sim alone to make a scene, put a robot in it, and attach ROS 2 to test your navigation stack, without touching reinforcement learning at all; or use Isaac Lab to draw directly on the dozens of training environments built into it, without having to build a scene from nothing.

Two: the Three Pillars

- PhysX: the physics solver, which handles rigid bodies, joints, contact and friction. What matters most for a robot is the quality of the contact solution—a foot on the ground and a gripper on an object both depend on it.
- RTX: ray-traced rendering, which produces the camera images. It is at the same time the foundation of simulating LiDAR and depth cameras (Section 33.4).
- OpenUSD: an open format for describing a scene, proposed by Pixar and by

now the standard for exchanging scenes between tools. Its value lies in one scene shared by several tools, and it also lets a robot model (links, joints, actuators, sensors) be described in a standard structure.

Three: the Important Changes of 2025

At SIGGRAPH on 11 August 2025, Isaac Sim 5.0 and Isaac Lab 2.2 became generally available, and the extensions of Isaac Sim were open-sourced on GitHub at the same time (isaac-sim/IsaacSim and isaac-sim/IsaacLab). For teaching and research this is a considerable matter: a simulator that could formerly be used only as a black box can now be read, altered, and submitted to.

One point must be made clear, lest students miss a step: the Omniverse Kit beneath it is not wholly open-sourced, and at present takes no outside submissions. Which is to say that you may modify Isaac Sim's extension layer and not the core at the very bottom. For a course project this is not usually a restriction, but it is worth knowing in advance when planning research for the long term.

The same release also brought several features that bear directly on robots: a robot structure description and an import wizard in OpenUSD, which makes a URDF conversion harder to get wrong; a model of joint friction, which lets the real resistance of a gearbox be written into the simulation; and support for ROS 2 Jazzy together with a standardized simulation interface, so that one set of ROS 2 nodes can be switched between different simulators (Part VIII).

What Isaac Lab 2.2 Brings

A cloud training environment that starts at one click (a Brev launchable), with no CUDA or drivers to install oneself.

The two-armed training environments of GR00T-Mimic, demonstrating learning manipulation from human demonstration data.

The integration of Omniverse Fabric, which lowers the memory and synchronization overhead of large scenes.

A tensorized suction gripper model, so that vacuum picking tasks may be simulated in parallel.

An open-source framework for evaluating policies, giving a trained policy a standard way of being compared (echoing Chapter 31).

33.3 GPU-Native Parallel Simulation

This is the technical heart of the whole ecosystem, and the question the first experimental figure of this chapter answers.

One: What GPU-Native Means

The procedure of a classical simulator (Gazebo, say) is: the CPU computes the physics → the state is copied to the learning program → the learning program computes an action → it is copied back to the simulator. Every step moves data between CPU and GPU, and once the environments are many the time spent moving data exceeds the time spent computing physics.

The GPU-native way is to keep the state of every environment as tensors directly in GPU memory, and to carry out the physics solution, the computation of observations, the computation of rewards and the inference of the neural network all on the GPU, so that the whole training loop never moves data back to the CPU. The difference cannot be seen when the environments are few, and at thousands of environments it is a

difference of orders of magnitude.

Two: What Parallelism Really Buys

Anyone can say that opening more environments makes it faster, but faster in what, faster by how much, and at what point it stops getting faster are three different questions. The experiment below, which can be reproduced in full, separates the three.

The experimental setting: a joint-tracking task with one degree of freedom. The joint has four parameters—rotational inertia, viscous damping, Coulomb friction and actuation delay; the controller is a set of PD gains with friction feedforward, which is to say three parameters to be learned. The training method is the plainest random hill climbing, each iteration evaluating the candidate parameters on N randomly sampled environments. The task is far simpler than real legged locomotion, and yet all three effects of parallelism show themselves fully here.

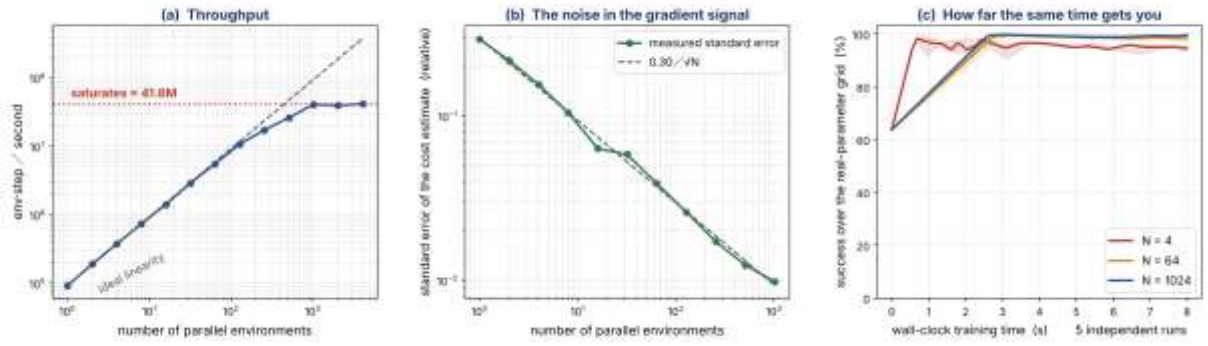


Figure 33-1 The Three Effects of the Number of Parallel Environments: Throughput, Gradient Noise, and the Stability of Training

Take (a) first. The horizontal axis is the number of environments N advanced together and the vertical axis the environment steps advanced per second, both on a logarithmic scale. The dashed line is ideal linearity—if parallelism were wholly free, doubling the environments should double the throughput. The measurement follows that line almost exactly while N is below about 32: from 93 thousand steps per second at N = 1

all the way to 2.91 million at N = 32, thirty-two times the environments bought with thirty-one times the throughput. The stretch is free because a single environment cannot fill the GPU at all, and the added environments merely put idle computing units to work.

Beyond N = 32 the curve begins to leave the dashed line, and by N = 4096 the throughput halts at about 41.8 million steps per second. Against N = 1 that is 452 times, and against the 379 million of ideal linearity only a ninth. The reason for the saturation is that the GPU's computing units are full already, and further environments can only queue. It is worth noting that from N = 1 to N = 4096 the wall-clock time of each batch rises only from 0.004 seconds to 0.039: the environments increase by a factor of 4,096 and the waiting by a factor of 9.8. Which is precisely the most immediate value of parallel simulation—samples had very nearly for nothing.

Panel (b) answers the second question: what those samples are for. Every update in reinforcement learning must estimate how good this set of parameters is, and that estimate, computed from the average over a finite number of environments, necessarily carries noise. The vertical axis is the standard error of that estimate, and the measured points fall almost perfectly on the dashed line of $1/\sqrt{N}$—multiply each point's standard error by $\sqrt{N}$ and the eleven results all lie between 0.25 and 0.33. From N = 1 to N = 1024 the standard error falls from 0.293 to 0.0098, close to the factor of $\sqrt{1024} = 32$. This is Formula One.

Panel (c) puts the first two together: given the same budget of wall-clock time (8 seconds), how far different degrees of parallelism get. To keep the luck of a single run out of it, each setting is trained five times independently; the heavy line in the figure is the median and the pale band the interquartile range. The median of N = 4 reaches

only 94.6 percent and goes no further, with an interquartile range of 93.3 to 95.8. The median of N = 64 is 98.3 percent and of N = 1024 is 99.6, the latter with an interquartile range of only 98.8 to 99.6.

What panel (c) is meant to convey is not that high parallelism is faster but that high parallelism is more reliable. N = 4 does not walk slowly; it walks erratically—the gradient noise is too large, many a step goes in the wrong direction, and the last few percent are never closed. The phenomenon is worse in real reinforcement learning, and it is why a framework such as Isaac Lab treats being able to open thousands of environments at once as a central measure.

An Honest Caveat

The three figures above were computed on a toy task with one degree of freedom, not on a legged robot. The absolute numbers of a real task will be wholly different: the point of saturation depends on your GPU and the complexity of the environment, and the number of environments needed depends on the dimension of the action and the sparsity of the reward.

But the three qualitative conclusions are general: throughput grows linearly and then saturates, gradient noise falls as $1/\sqrt{N}$, and with too little parallelism the variation in the outcome of training is too large to compare. Drawing the same three figures for your own task is the most effective way of planning computing resources.

Three: the Numbers at Real Scale

For comparison, look at figures at industrial scale. The Isaac Lab technical report (November 2025), on a configuration of 8 GPUs with 16,384 environments each, reports

the teacher policy of a dexterous-hand grasping task reaching more than 900 thousand steps per second, and a Franka arm cabinet-opening task more than 1.6 million, with near-linear scaling across several machines and cards.

Another figure more practical for students: on the cabinet-opening task the throughput of one consumer-grade GeForce 5090 is about equal to two professional cards, and on the dexterous-hand task it is even about 25 percent faster, because the bottleneck of that task is the CPU and not the GPU. Which means that a desktop machine with a single high-end graphics card is enough to support quite serious research in robot learning—something unimaginable five years ago.

Four: Formula One—Parallelism and the Quality of an Estimate

Let the estimate of the return from a single environment be a random variable with standard deviation σ; taking the average over N independent environments as the estimate, the standard error of the estimator is:

$$SE(N) = \sigma / \sqrt{N}$$

To lower the noise of the estimate to a tenth requires a hundred times the environments. This penalty of the square root explains why parallelism is counted in thousands: raising the environments from 64 to 4,096 lowers the noise only to an eighth. It explains at the same time why the saturation of panel (a) is acceptable—even with the throughput down to a sixth of the ideal, the added environments are still lowering the noise.

33.4 Simulating Sensors: from RTX LiDAR to Touch

For the reader of this book this section is probably the closest to home: what does every sensor used in the past thirty-two chapters look like in simulation?

One: the Four Families of Sensor

Family	How it is implemented in simulation	Corresponding chapters	Quality of the imitation
Proprioception	Read directly from the physical state	Chapters 7, 10	High
Camera	RTX ray-traced rendering	Chapters 9, 20–23	Fairly high
Ranging	Warp ray casting	Chapters 8, 17–18	Medium
Contact and touch	The contact solver + a vision-based tactile model	Chapter 19	Fairly low

Table 33-3 The Four Families of Sensor in Simulation

Two: One by One

Proprioception is the easiest and is imitated the best. The IMU, the joint encoders, contact detection and the frame transformer can all be read straight off the physical state, with bias and noise added by the error model of Chapter 7. But note: what the simulator gives is a clean truth plus the noise you wrote, while a real IMU's error has temperature drift and vibration coupling in it, and is not independent identically distributed Gaussian noise.

The camera is where RTX rendering is at home. Besides RGB, depth, semantic segmentation, instance segmentation, surface normals and motion vectors all come free—data that would need extra labeling in the real world are a by-product of the rendering pipeline in simulation. This is precisely the technical foundation of the synthetic data of Section 33.5. The lens model supports both pinhole and fisheye, the

latter mattering for the wide-angle visual navigation mentioned in Chapter 9.

Ranging sensors (LiDAR, and the stereo matching of a depth camera) use Warp ray casting: rays are cast from the sensor origin in the specified scan pattern and their intersections with the scene computed. This handles occlusion and geometry correctly and can imitate the different beam arrangements of various LiDARs; the terrain height scan that legged robots commonly use is implemented by the same mechanism. But two things it does not easily get right: the specular reflection and transmission of Chapter 8 (glass, water, a highly reflective sign), and the motion distortion of Chapter 18—the latter needing the simulator to keep updating the sensor pose during the scan, rather than casting every ray at one instant.

Contact and touch are at present the class with the largest gap. The contact force itself is solved by PhysX, but real contact involves the microscopic deformation of the material, the roughness of the surface and viscous effects, which a rigid-body solver can only approximate by a coefficient of friction. The simulation of vision-based tactile sensors (which infer contact force from a camera's view of the deformation of an elastic membrane) is usable by now, but if a student's project is to do fine insertion or assembly, the sim-to-real gap of that class of task must be expected to be the largest.

A Detail Often Overlooked: the Timing of the Sensors

Real sensors do not trigger together. A LiDAR takes 100 milliseconds to turn once, a camera's exposure a few milliseconds, the IMU runs at 200 Hz and the wheel encoders at 50—all discussed in Chapters 7 and 18.

A simulator by default often treats every sensor as sampling at the same instant. If your algorithm is sensitive to time synchronization (a tightly coupled

LiDAR-inertial odometry, say), then fine in simulation and drifting on the real machine is quite likely not a fault of the algorithm but of a simulation that did not build the time differences in.

The remedy is to set the update rate and the phase of every sensor explicitly in simulation, and to add random jitter in time deliberately.

33.5 Generating Synthetic Data and Reconstructing a Real Scene

Chapter 32 discussed the necessity of collecting one's own data, and the cost of labeling. This section offers another road: let the simulator produce the data and the labels together.

One: the Temptation of Synthetic Data, and Its Trap

The temptation is plain: in simulation it is known which object every pixel belongs to, so that segmentation labels, bounding boxes, depth maps and six-degree-of-freedom poses are all free and without error. Ten thousand training images under different lighting, from different viewpoints, with objects differently arranged, are a script and a few hours of computation away.

The trap is just as plain: a simulated image is not the same as a real one. The reflectance of materials, the noise and compression artifacts of a sensor, the dirt and color cast of a lens, the clutter in a real scene that nobody would trouble to model—these differences together are called the domain gap. A model trained on synthetic data alone will often do poorly on real images.

Two: Three Roads to Narrowing the Domain Gap

1. Reconstructing the real scene: do not model it yourself, but scan the real scene in. The present practice is to reconstruct an interactive three-dimensional scene from real video (the NuRec library and the open-source 3DGUT method, for instance), so that the background and the materials are real to begin with and only the dynamic objects are synthetic.
2. Visual domain translation: use a generative model to translate a simulated image into one that looks as though it were really photographed. Isaac Sim's data writer can already be connected directly to models of this kind (Cosmos Transfer, say).
3. Visual domain randomization: the reverse—do not imitate reality, but randomize the simulated materials, lighting and background to a degree so extreme and exaggerated that the model has no choice but to rely on the shape of the object rather than its appearance. This road is the simplest and the most used.

Three: Mixing Synthetic and Real

What works best in practice is usually not one or the other but a mixture: pre-train on a great deal of synthetic data and fine-tune on a little real data. This echoes a point made in Chapter 32—fine-tuning from pre-trained weights needs far less real data than training from nothing. A practical recipe for a campus project is: several thousand synthetic images for pre-training, 200 to 500 real images per category for fine-tuning, and a test set made of real data only.

Figure 33-2 The Iterative Flow of Data from Collection to Deployment

① Define the categories and plan the collection

↓

② Obtain the data (really photographed / generated in simulation / a mixture of both)

↓

③ Label (set a standard for the real data and spot-check it; synthetic data are labeled automatically)

↓

④ Split the dataset (split by place rather than at random; the test set is real data only)

↓

⑤ Train (pre-train on synthetic data → fine-tune on real data)

↓

⑥ Assess (compute the accuracy on the real test set; and test on the real site)

↓

⑦ Analyze the errors (which categories go wrong most often? under what conditions?)

↓

⑧ Add data aimed at the weak points → return to ⑤ and iterate

↓

⑨ Optimize for deployment (quantize, accelerate) → the edge device (Section 33.8)

Note the seventh and eighth steps particularly: analyzing the errors and adding data aimed at them is often far more effective than blindly adding training epochs. Synthetic data are especially valuable at this step—when you find the model goes wrong most often on a staircase against the light, you can generate a thousand backlit staircases in simulation on purpose, which in the real world would mean waiting on the weather.

Two Tools Worth Knowing

MobilityGen: generates trajectories for a mobile robot and the corresponding

sensor data automatically, suited to producing datasets for training or assessing SLAM and navigation.

Describing a scene automatically with a vision-language model: once the simulator has rendered an image, a VLM produces a textual description as the label—which joins the semantic labeling of Chapter 32 to the synthetic data of this chapter, and is one of the chief sources of the language navigation data of Chapter 34.

33.6 Domain Randomization and the Sim-to-Real Gap

This is the most important section of the chapter. The five sections before it were about how to train in simulation; this one is about why it fails when carried to a real machine once trained, and what to do about it.

One: Where the Gap Comes From

Source of the gap	What it consists of	Where it appears in this book
Actuator dynamics	The virtual inertia created by firmware compensation, the bandwidth of the current loop, torque saturation	This section
Error in link inertia	The CAD model underestimates the real inertia (cabling, shells, sensors)	This section

Friction and damping	The Coulomb friction of a gearbox and damping that changes with the state	This section
Time delay	The total delay caused by command transmission, filtering and the control period	Chapter 25
Joint offset	The zero offset caused by errors of assembly and calibration	Chapter 10
Sensor characteristics	The noise model, the sampling time, the field of view and occlusion	Chapters 7–9
Differences in the environment	Ground friction, terrain, load, temperature and ageing	Chapter 19

Table 33-4 Seven Sources of the Sim-to-Real Gap

Two: the First Remedy—System Identification

The first road is to make the simulation right: measure the real robot's response and work back to the parameters the simulation ought to use. The standard practice is to hang the robot in the air (so that the feet touch nothing and contact forces do not interfere), send each joint a frequency sweep (a sine wave whose frequency rises continuously from low to high), record the encoders' response, and work back to the parameters by optimization. Recent research notes that about 20 seconds of sweep data in the air per robot is enough, with about four parameters per joint to be identified (armature inertia, viscous damping, Coulomb friction, joint offset) plus one global

command delay.

The procedure is demonstrated below with an experiment that can be reproduced in full. The real robot is a joint model with Coulomb friction and a 7.5-millisecond command delay; the URDF model has only the inertia of the links—no damping, no friction, no delay—which is precisely the typical state of a URDF exported straight from CAD.

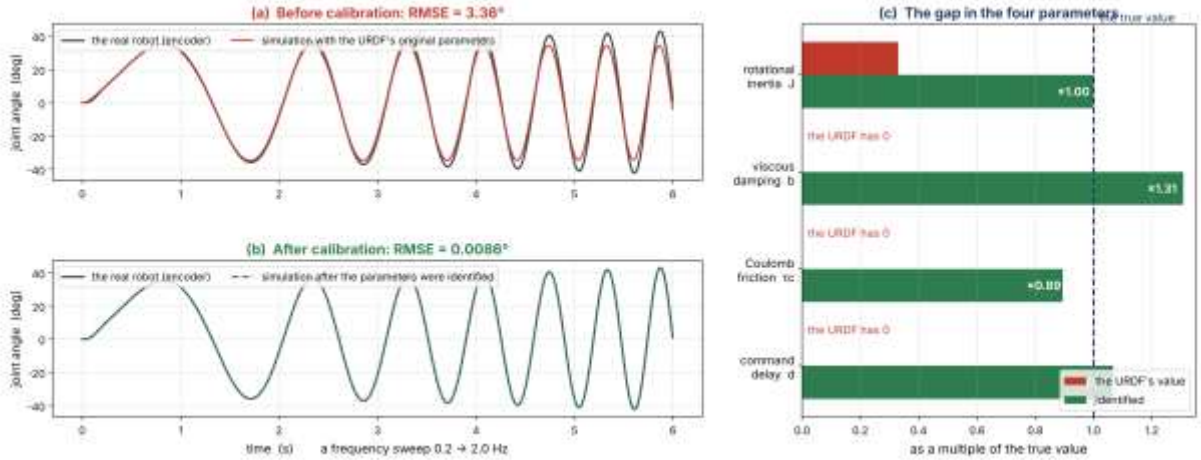


Figure 33-3 System Identification: from the Guess in the URDF to the Fitted Simulation

Panel (a) is before calibration. The grey line is the angle recorded by the encoder of the real robot, and the red the same motion simulated with the URDF parameters. The two curves are alike in shape, but the red plainly leads—the consequence of the delay not being modeled; its amplitude is slightly larger too, because nothing in the simulation is consuming energy by friction and damping. The root-mean-square error over the whole stretch is 3.36 degrees. For a walking policy, a systematic bias of three degrees and more at every joint is enough to throw the gait entirely out of shape.

Panel (b) is after calibration. The green dashed line lies exactly over the grey, and the root-mean-square error falls to 0.0086 degrees, a four-hundredth of what it was. It must be stressed that the model was not made more complicated: the parameters of

the same model were set right, and the form of the equations did not change from beginning to end.

Panel (c) shows how far the parameters were out. The red bars are the URDF's original values as a multiple of the true ones and the green bars the identified results. In rotational inertia the URDF holds only 0.33 of the true value—the missing two thirds coming from the virtual inertia created by the firmware's compensation and from cabling and shells that the CAD did not count. Viscous damping, Coulomb friction and command delay are simply zero in the URDF. After identification the inertia is restored exactly, to 1.00 of the true value; the delay, searched on a grid at intervals of 2 milliseconds, lands at 8.0 milliseconds against a truth of 7.5, and damping and friction come out at 1.31 and 0.89 times the truth—they have absorbed between them the half grid point by which the delay is out. Which is the compensation among parameters that the next subsection warns of, seen here in miniature.

A Pitfall in Implementation: the Delay Cannot Be Fitted by a Gradient Method

In a discrete-time simulation the command delay is an integer number of time steps, so the residual is a step function of the delay—and within each step the gradient is zero. Throw the delay into a gradient method such as least_squares and it will sit at its initial value without moving.

The right course is to search the delay on an integer grid in the outer loop and fit the continuous parameters by a gradient method in the inner one. The first time the experiment of this section was done wrongly, the delay fitted came out at 0.0001 seconds (the truth being 0.0075), and the inertia was overestimated by a factor of 2.3 to make up for the delay that had not been

identified—the trajectories looked right and the parameters were all wrong. This is the most characteristic trap of system identification: a small error does not mean the parameters are right, and different parameters may compensate for one another.

Three: the Second Remedy—Domain Randomization

The second road is the reverse: do not pursue a correct simulation, but make the policy immune to the uncertainty of the parameters itself. The method is to draw the dynamic parameters of every environment at random from a distribution during training, forcing the policy to learn a behavior that works for any reasonable parameters.

The question is: how wide should the randomization be? The experiment below compares three settings. On the same joint-tracking task, three controllers are trained: (a) on the nominal parameters alone; (b) randomized within ±10 percent of the nominal values; and (c) randomized over a wide range—inertia from 0.5 to 2.0 times, friction from 0.3 to 2.0 times, delay from 0 to 28 milliseconds. The three are then assessed on the same grid of real parameters.

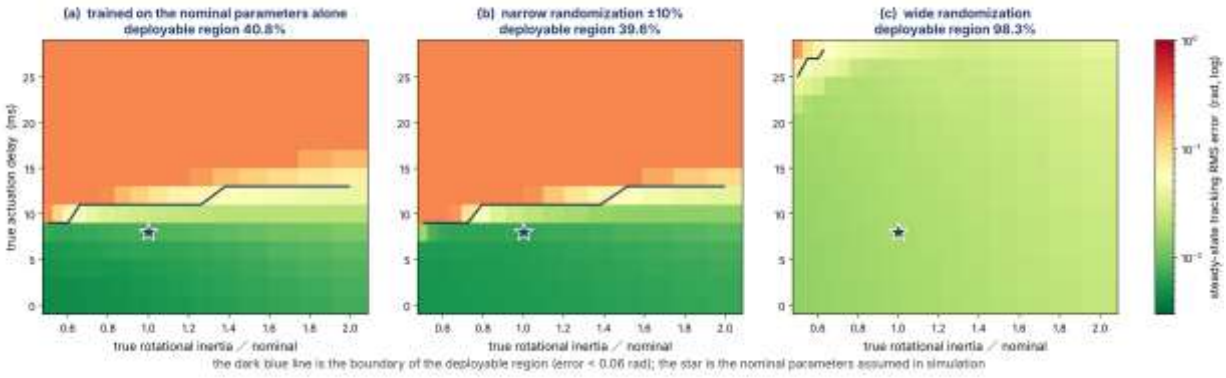


Figure 33-4 How the Range of Domain Randomization Affects the Rate of Successful Deployment

Take the plainest conclusion first: the deployable region (the area enclosed by the dark

blue line) leaps from 40.8 percent in (a) and 39.6 in (b) to 98.3 in (c). Narrow domain randomization is of almost no help—a range of ±10 percent is too narrow, and the policy learned is much the same as one trained at the nominal point alone. The point is worth students remembering: domain randomization is not a matter of adding a little randomness, and the range must be wide enough to cover the uncertainty you really have about the real parameters.

The second conclusion is hidden in the colors. Within the deployable region (a) and (b) are deep green, meaning a very small tracking error; (c) is pale green all over, meaning it works everywhere and is nowhere good enough. Quantitatively: at the nominal parameters the controller trained on the nominal parameters alone has an error of 0.0069 radians and the widely randomized one 0.0179—two and a half times as much. This is what domain randomization costs: robustness is not free, and it is bought with nominal performance.

The third conclusion is the shape of the boundary of failure. The boundary in (a) and (b) is nearly a horizontal line, falling at a delay of between 9 and 13 milliseconds, and rising only slightly with the rotational inertia. Which is to say that in this task being out by a factor of two in inertia will not make the policy fail, while a few milliseconds more of delay will. The message is of practical use in planning a project: rather than spend effort weighing every link, measure the delay of the control loop first.

Four: How to Choose Between the Two Roads

	System identification	Domain randomization
The method	Measure the real machine and make the simulation	Leave the simulation alone and train the policy to take

	right	punishment
What it needs	A real machine that can be measured, and equipment to excite it	An estimate of the range of uncertainty in the parameters
Its merits	High nominal performance, and explicable	No real machine needed in order to train
Its faults	The parameters may compensate for one another; it fails as parts wear	Nominal performance falls; the range is hard to set
It suits	Having a real machine already, and wringing the best performance out of it	The real machine not yet in hand, or a fleet whose machines differ greatly

Table 33-5 System Identification Compared with Domain Randomization

In practice the two are usually used together: calibrate the center of the simulation to the right place by system identification first, then randomize moderately about that correct center to cover wear, temperature and the differences between individual machines. This neither sacrifices nominal performance as pure domain randomization does, nor fails once a part has worn as pure system identification does.

33.7 How to Begin, and How to Join the Community

This section is very practical: what a student who has read the six sections above and wants to begin should do next.

One: Three Routes In

Route	What it needs	Whom it suits
A one-click start in the cloud	A browser and an account	Students with no graphics card of their own
Deployment in a container	A Linux host with CUDA + a container environment	A laboratory with a workstation
Installing from source	The same, and handling the package dependencies oneself	Those who are to modify the simulator

Table 33-6 Three Routes In

For most students, beginning from a one-click start in the cloud is advised. It settles the driver, the CUDA version and the package dependencies—the things most apt to hold one up—so that you can see a robot moving in simulation on the first day instead of spending three days installing an environment. Once the procedure is familiar and you are sure of committing to it for the long term, move to a local installation.

One detail is worth knowing before a local installation: the simulator's display is streamed out over WebRTC, using ports 49100 and 47998 by default. If you cannot reach the display on a school network, nine times in ten the firewall has blocked those two ports and the software is not broken.

Two: A Suggested Order of Learning

1. Get the built-in example running first: change nothing, and confirm that your environment can train a robot that walks. The point of this step is only to establish a baseline.
2. Change the robot and not the task: replace the robot in the example with another built-in model, and observe how the same rewards perform on a

different morphology. The framework has about eleven legged morphologies built in, enough to compare for a long while.

3. Change the rewards: this is the step from which most is learned. Raise the weight on velocity tracking, raise the penalty on energy, and observe how the gait changes. You will soon feel that designing rewards is a craft.

4. Import your own robot: convert your own URDF into a form the simulation can use with the import wizard, at which point the real problems of joint limits, inertia and friction settings begin.

5. Attach ROS 2: let the simulator serve as the test bench of your navigation stack (Part VIII). This step needs no reinforcement learning, and is of the greatest value to students working on SLAM.

6. Do system identification and sim-to-real: once a real machine is at hand, go once through the procedure of Section 33.6.

Three: How to Join the Community

Robot simulation is a field that changes quickly, and the documentation often lags behind the code. Knowing where to ask is therefore nearly as important as knowing how to do it.

- The developer forum: the official robotics developer forum is the most effective place to ask, and the engineers reply directly. Attach the version number, the model of the graphics card and the complete error message, and the chance of a useful reply is much higher.
- GitHub: the code of Isaac Sim and Isaac Lab is public on GitHub. When behavior and documentation disagree, reading the source is often quicker than searching. Issues and changes may be submitted too—the most substantial way of taking

part.

- Official courses and newsletters: the free introductory robotics course suits students with no background at all, and the robotics newsletter is the least effortful way of keeping up with changes between versions.
- Papers: the figures in technical reports and papers (such as the throughput cited in Section 33.3) are the most reliable basis on which to plan a hardware purchase, and far more accurate than marketing material.

Three Reminders for the Project Student

Pin the version. This ecosystem updates quickly and the API changes between versions. Write the version number into the documentation when the project begins, and have the whole team use the same one.

Do not pursue perfection in simulation. The value of simulation is in iterating quickly, not in realism at the level of the pixel. Three weeks spent making the materials beautiful usually does nothing whatever for the sim-to-real success of the policy.

Think out what you are going to measure first. Training a robot that walks is only the starting point; the value of a project lies in what you measured and what you compared (Chapter 31).

33.8 Deployment at the Edge: Jetson, Quantization and Distillation

Training happens in the cloud or on a workstation, and execution on the robot. This section takes up the last leg of that train and deploy chain: how to fit the model into a computing module the size of a palm and running on a battery.

One: the Scale of the Problem

The computational demands of a vision-language model usually confine it to the cloud, and running in the cloud brings every problem discussed in Section 25.1: dependence on the network, latency, bandwidth and reliability. For a robot working in an underground car park or a factory, stopping dead when the network drops is not an acceptable design.

Two: Choosing the Hardware

Module	AI performance	Memory and bandwidth	Power	Typical use
AGX Thor T5000	2,070 TFLOPS (FP4 with sparsity)	128 GB, 273 GB/s	40–130 W	Running a large model aboard
AGX Orin 64GB	275 TOPS (INT8)	64 GB, 204.8 GB/s	15–60 W	SLAM + several models in parallel
Orin NX 16GB	100 / 157 TOPS	16 GB, 102.4 GB/s	10–40 W	The mainstream of inspection robots
Orin Nano 8GB	40 / 67 TOPS	8 GB, 68 GB/s	7–25 W	Teaching and student projects

Table 33-7 Edge Computing Modules Compared

Two traps are to be avoided in reading this table. First, Thor's figure is in TFLOPS at FP4 with sparsity and the Orin series' in TOPS at INT8, so the two cannot simply be divided one by the other—a figure seven times larger does not mean performance seven times

better. Second, and more important: for a robot running SLAM and an AI model at once, what decides whether it can run at all is often not the column of computing power but the column of memory. Case 33-2 explains this.

Three: the Three Key Techniques

Quantization: representing the model's parameters in fewer bits. Vision-language and vision-language-action models at the edge at present take 4-bit weights with 16-bit activations as their default configuration; a format of 4-bit weights with 4-bit activations unlocks the newest generation of hardware platforms and is the production-grade upgrade where it is supported; and 8-bit weights with 8-bit activations remain the safer fallback for mature vision and segmentation models.

Distillation: transferring the ability of a large model to a small one. Research has proposed a distillation strategy that transfers the complex spatial-semantic reasoning of a large frontier model to a light local model of 4 billion parameters, so that it may run on an embedded GPU device; the experiments show that fine-tuning the small model on about 500 reasoning traces from the frontier model is enough to give it navigation ability, with a success rate of 34.5 percent against the 39.7 of the large cloud model—quite close.

Memory management: a link often overlooked and extremely important. Research running SLAM and a VLM together on a Jetson Orin Nano found that a map management strategy founded on geometry suffered out-of-memory failures and stalls in execution; after changing to a key-frame loading and unloading mechanism partitioned by semantics, it not only did away with both problems and held the localization steady but raised the throughput and lowered the latency by 21.7 percent.

Four: Formula Two—Estimating the Memory After Quantization

The memory a model needs after quantization is approximately:

$$\text{memory} \approx \text{number of parameters} \times \text{bits per parameter} \;/\; 8 \quad \text{(bytes)}$$

Take a model of 4 billion parameters: 16-bit precision needs about 8 GB, 8-bit about 4 GB, and 4-bit about 2 GB. This explains why 4-bit quantization can squeeze a large model onto an embedded platform—the difference is fourfold. Set against Table 33-7: a 4-bit quantized model of 4 billion parameters takes about 2 GB, and on an 8 GB Orin Nano it must share the remaining 6 GB with the operating system, SLAM's map and the image buffers, which is already quite tight.

But note: the memory actually used includes activations, intermediate results and the overhead of the framework as well, and is usually 1.2 to 2 times the estimate above. Leave margin when planning.

Five: Three Remedies

1. Limit the size of the map: load only the region currently needed, partitioned by semantics. This echoes the multiple maps of Atlas in Chapter 23 and the submaps of Cartographer in Chapter 15—cutting a large problem into manageable pieces.
2. Choose a small quantized model: there is no need to carry one large model for every task. One quantized general model is often more practical than three unquantized specialized ones.
3. Stagger the execution: AI inference need not run on every frame. Once or twice a second is usually enough to support decisions at the semantic level, while SLAM's front end keeps its original rate.

A Checklist Before Deployment

Have you measured the peak memory on the target hardware? (Not the average—the peak.)

Have you run SLAM and the AI model together? Both being fine apart does not mean they are fine together.

Have you measured the end-to-end latency? From the image coming in to the decision going out, not merely the model's inference.

Have you tested with the network down? If there is any dependence on the cloud, this must be tested.

Have you tested running for a long time? A memory leak and the growth of the map take hours to show themselves (Section 19.6).

Cases

Case 33-1 With an Uncalibrated Simulation, the Policy Trained Falls Over at Once

This is the real experiment that states the system identification of Section 33.6 most clearly.

A research team did the same thing to several legged robots: hung the robot in the air, sent its joints a frequency sweep lasting 20 to 60 seconds, and, using the encoder data alone, fitted the dynamic parameters by evolutionary optimization over 4,096 parallel environments. The parameters to be identified were the armature inertia, viscous damping, Coulomb friction and zero offset of every joint, plus one global command delay—about forty-nine in all.

The crucial controlled comparison was this: a policy trained with the uncalibrated URDF parameters, placed on a real quadruped, failed even to walk forward and had to be caught by the hoist; while a policy trained with the identified parameters walked normally. The same algorithm, the same rewards, the same training procedure—the difference lay only in whether the simulation's parameters were right.

The team quantified the sources of the gap at the same time, and every one answers to Table 33-4: the firmware's compensating behavior created a virtual inertia of about 8.1×10^{-3} kg·m², the CAD underestimated the inertia of the thigh link, the global command delay was about 7.5 milliseconds, and the zero offset of a joint reached about 5 degrees at most. Not one of these figures can be read off a CAD drawing.

Calibration brought more than being able to walk. The cost of transport measured by the same procedure on different models fell from 1.86 to 1.27, an improvement of about three tenths in energy; as for endurance, one quadruped covered 4.12 kilometers on a single charge and another 6.10; and a small quadruped of 4.2 kilograms climbed twenty stairs of 18 centimeters each in 46 seconds.

There are two lessons for students. First, about 20 seconds of data suffice for the identification—an experiment a one-semester project can well afford. Second, and more important: after identification the team used no domain randomization of the dynamics at all. This agrees with the conclusion of Figure 33-4 in Section 33.6—once you can calibrate the simulation accurately enough, there is no need to buy robustness with nominal performance. Domain randomization is the insurance for when you cannot calibrate, not the default.

Case 33-2 Memory Is the Real Bottleneck

This is a problem many students run into only late in a project, and knowing of it in advance saves a great deal of frustration.

The situation is this: the student has SLAM running successfully on a Jetson and the object detection model running successfully too, but the moment the two run together the system becomes extremely slow, or crashes outright. And yet a check of the processor and graphics processor shows neither fully loaded—which is puzzling.

The real bottleneck is memory. Research has quantified the phenomenon clearly: running SLAM and a vision-language model together on a Jetson Orin Nano, a map management strategy founded on geometry met out-of-memory failures and stalls in execution; while a key-frame loading and unloading mechanism partitioned by semantics not only did away with both and held the localization steady but raised the throughput and lowered the latency by 21.7 percent.

The root of the problem is that SLAM's map keeps growing as exploration goes on (the memory estimate of Section 13.5, the number of particles times the size of the map of Chapter 14), while the AI model takes a great deal of memory in itself. With the two in contention the system is bound to fail.

The ingenuity of the solution lies in partitioning by semantics: the robot need not remember the detail of a whole building at every moment, but only load the map of the region it is in. The idea descends directly from the multiple maps of Atlas in Chapter 23 and the submaps of Cartographer in Chapter 15—cutting a large problem into manageable pieces.

A concrete piece of advice for a project: estimate the memory needed by Formula Two at the planning stage, and consider a mechanism for limiting the size of the map first.

Dealing with it only once the system crashes usually means a large restructuring.

Case 33-3 Putting a Large Model into a Robot: the Power of Distillation

This case answers the doubt students are most likely to have: if a large model will not run, is there nothing for it but to give up?

The background is plain enough: vision-language models have become a powerful tool in robotics, able to understand an environment in the context of language, and have given rise to open-vocabulary tasks such as object-goal navigation; but their computational load usually confines them to the cloud.

What the research team did was to distill: first build a high-quality pipeline founded on a scene graph with a large frontier model, reaching a success rate of 39.7 percent on the standard open-vocabulary object navigation benchmark; then transfer that reasoning ability to a local model of only 4 billion parameters, so that it might run on an embedded GPU device.

The crucial result is this: fine-tuning the small model on about 500 reasoning traces from the frontier model alone was enough to give it navigation ability, with a success rate of 34.5 percent, narrowing the gap to the large cloud model greatly.

Five hundred pieces of data—the figure is worth savoring. Set against the hundreds of thousands of labels needed to train from nothing, the cost is tiny. And what it buys is a change in kind, being able to run on the robot: no network needed, no cloud latency, and no data leaving the machine.

The methodological lesson of this case agrees exactly with the NanoSLAM case of Chapter 25: the bottleneck in performance is often not a want of computing power but

a method not yet optimized. When you feel that this technique is too heavy and will not run, the first question should not be whether to give up but whether there is a cleverer way.

Implementation: from Training in Simulation to Deployment at the Edge

The implementation of this chapter falls into three stages of increasing difficulty. Stage one needs no special hardware; stage three needs an embedded computing module.

Stage One The Three Figures of Parallelism (no robot needed)

Reproduce the three measurements of Figure 33-1 of this chapter. The task may be of your own choosing; the simplest is the single-joint tracking used here, though any vectorized environment you are familiar with will serve.

1. Measure the throughput: fix the policy, run one batch for each of N = 1, 2, 4, …, 4096, record the wall-clock time and convert it to environment steps per second. Draw it on logarithmic axes and mark the point of saturation.
2. Measure the gradient noise: fix the policy, sample forty times over for each N, and compute the standard deviation of the cost estimate. Verify whether it falls as 1/√N (multiply by √N and see whether it is nearly constant).
3. Measure the stability of training: fix the budget of wall-clock time, run five independent trainings for each N, and draw the median and the interquartile range. What matters is not whose median is higher but whose interquartile range is narrower.

The product of this stage is a parallelism curve for your own hardware. It will tell you at what number of environments it stops being worth it on your machine—the basis on

which every training experiment afterwards is planned.

Stage Two System Identification and Domain Randomization (one joint with an encoder needed)

The least that is required is a servo motor or gearbox module with an encoder, able to take a position command and report the actual angle. One joint of a robot arm will do.

1. Record the sweep: let the joint carry no load, send a swept position command from 0.2 to 2 Hz, record the command and the encoder reading at as high a rate as possible, for 20 to 60 seconds.
2. Build the model: write down a discrete-time model with rotational inertia, viscous damping, Coulomb friction and a command delay. Start with the CAD value of the inertia and zero for the rest, simulate once and compare with the measurement—that is your Figure 33-3(a).
3. Do the identification: search the delay on an integer grid in the outer loop and fit the continuous parameters by least squares in the inner one. Remember to verify whether the parameters compensate for one another: verify again on a different excitation signal, and if the parameters run away when the signal changes, the identification is not yet steady enough.
4. Measure the cost: train one set of controllers with the identified parameters and another with wide-range domain randomization, and compare their tracking error on the real joint. You should see the same phenomenon as in Figure 33-4: the randomized set takes more punishment and is less accurate at the nominal point.

Stage Three Deployment at the Edge (an embedded computing module needed)

1. Establish a baseline: run SLAM alone on the module and record the peak memory, the latency and the update rate. Then run your AI model alone and record the same three.
2. Run them together: run both at once and measure the three again. Compare the combined figures with the sum of the separate ones; the difference is the loss caused by contention.
3. Quantize: quantize the model to 8 bits and to 4, and measure the memory and the accuracy of each. Set the estimate of Formula Two against the measurement and see how much of it is the framework's overhead.
4. Apply the remedies: try the three remedies of Section 33.8 one by one, changing one thing at a time and measuring again, and build a table of remedy against effect.

The product of this stage is a deployment report made of figures you measured yourself. It carries more conviction than any specification sheet, and it is the most practical close to this book's technical line.

Further Reading

- The Isaac Lab technical report (2025). (The architecture and throughput figures of GPU-native parallel simulation, the basis of Section 33.3.)
- The GitHub projects and official documentation of Isaac Sim and Isaac Lab. (The practical foundation of Sections 33.2 and 33.7.)
- Research on systematic sim-to-real transfer for legged robots (2025). (The source of Section 33.6 and Case 33-1.)
- Survey papers on learned legged locomotion control. (The background to

Section 33.1, with comparisons among the various methods.)

- The OpenUSD specification and the robot structure description documentation. (A reference for the scene and robot description formats of Section 33.2.)
- NVIDIA Jetson developer documentation and the inference optimization guide. (The technical foundation of the edge deployment of Section 33.8.)
- The edge computing of Chapter 25, the methods of assessment of Chapter 31, and the data collection and labeling of Chapter 32. (The direct joints between this chapter and the rest of the book.)

The Points of This Chapter

- A reinforcement-learned robot policy needs interactions with the environment of the order of a billion, which on a real machine comes to about 231 days of continuous running, so that training must be done in simulation first. Simulation offers accelerated time, failure at no cost, a wholly known state, repeatable experiments and environments that can be generated; but it solves the equations you wrote into it, not real physics.
- Isaac Sim is the simulator and the scene environment (PhysX physics, RTX rendering, OpenUSD scenes) and Isaac Lab the robot learning framework built upon it (tasks, observations, rewards, the training procedure). Both became generally available in August 2025 with their extension layers open-sourced, though the Omniverse Kit beneath is not wholly open-sourced and takes no outside submissions.
- The key to GPU-native simulation is that the state lives in GPU memory as tensors, so that the whole training loop never goes back and forth to the CPU. The measurement shows throughput nearly linear while $N \leq 32$ (from 93

thousand steps per second to 2.91 million), saturating thereafter at about 41.8 million; and yet with the environments increased 4,096-fold the waiting per batch rises only 9.8-fold.

- The real value of parallelism is in lowering gradient noise. The measured standard error of the cost estimate falls precisely as 1/√N (multiplied by √N, all eleven points lie between 0.25 and 0.33). Within the same budget of 8 seconds, the median success of N = 4 is only 94.6 percent with an interquartile range spanning 93.3 to 95.8, while N = 64 reaches 98.3 and N = 1024 reaches 99.6 with an interquartile range of only 98.8 to 99.6—with too little parallelism the training stops short and its outcome varies more widely.
- Sensor simulation falls into four families: proprioception (high), camera (fairly high), ranging (medium) and contact and touch (fairly low). The camera's segmentation, depth and normals are by-products of rendering, which is the technical foundation of synthetic data; while specular reflection in LiDAR, motion distortion, and the time differences between sensors are the sources of the gap most often overlooked.
- The temptation of synthetic data is that the labels are free and without error, and the trap is the domain gap. The three roads to narrowing it are reconstructing the real scene, visual domain translation and visual domain randomization; what works best in practice is pre-training on synthetic data with fine-tuning on a little real data, with a test set of real data only.
- The sim-to-real gap has seven sources, of which actuator dynamics, errors in link inertia and time delay are most often underestimated. The system identification experiment shows the URDF's rotational inertia to be only 0.33 of the true value, with damping, friction and delay simply zero; after calibration the

root-mean-square error between simulation and reality falls from 3.36 degrees to 0.0086, about a four-hundredth.

- System identification has one characteristic trap: in discrete time the delay is a step function, so that a gradient method stops dead at its initial value while the inertia is overestimated to compensate—the trajectories agree and the parameters are all wrong. The right course is to search the delay on a grid in the outer loop.
- The range of domain randomization must be wide enough to cover the real uncertainty: a narrow ±10 percent is of almost no effect (a deployable region of 39.6 percent against 40.8 with no randomization), while a wide range reaches 98.3. The cost is that the tracking error at the nominal point rises from 0.0069 radians to 0.0179, about two and a half times—robustness is bought with nominal performance.
- The measured boundary of failure is decided almost entirely by the delay (about 9.5 milliseconds) and is nearly independent of the rotational inertia. Which shows that in planning, the delay of the control loop should be measured before the mass of every link is computed precisely.
- The two remedies are usually used together: calibrate the center of the simulation correctly by system identification first, then randomize moderately about it to cover wear, temperature and the differences between individual machines. Case 33-1 shows that when the calibration is good enough, domain randomization of the dynamics may be dispensed with entirely.
- The three great techniques of edge deployment are quantization (4-bit weights are at present the default for vision-language models at the edge), distillation (fine-tuning a 4-billion-parameter model on about 500 reasoning traces reaches

34.5 percent, close to the 39.7 of the large cloud model), and memory management (key-frame loading and unloading partitioned by semantics does away with out-of-memory failures and lowers the latency by 21.7 percent).

- In choosing an edge module, what decides whether it can run is often the column of memory rather than the column of computing power; and different generations are labeled in different units (TFLOPS at FP4 with sparsity cannot be compared directly with TOPS at INT8). The memory actually used is about 1.2 to 2 times the estimate of Formula Two.

Carrying Forward

This chapter has carried the techniques of the book from running to learning: Chapter 32 gave the map a meaning, and this chapter has given the robot a way of acquiring a skill in simulation and of sending the trained model all the way to the computing module on its body.

But one piece of the puzzle is still missing. What this chapter trains is a policy for how to walk, and what Chapter 32 built is a map of what is here—between them is missing why go there. When the user says go to the meeting room and turn off the projector, the robot must break one sentence into a succession of executable subgoals, must reason in an environment it has never seen about where a meeting room usually is, and must know where to explore when it cannot find the goal.

That is the subject of Chapter 34: vision-language models, language navigation, and embodied intelligence. It is also the last chapter of the book—we shall gather up the threads from Chapter 1 to Chapter 34 and use one complete task to explain where each of these thirty-four chapters plays its part.

Taiwan-Authored University Textbook

@prof

SLAM

Simultaneous Localization and Mapping, SLAM

Chapter 34

VLMs, Language Navigation and Embodied Intelligence

Editor: Lee Shihyuan (李世淵) LINE OA @prof

Assistant: Aaron Lee, Anne Lee

University Intelligent Robotics Program / High School AI and Robotics Club Course

Where This Chapter Sits in the Book

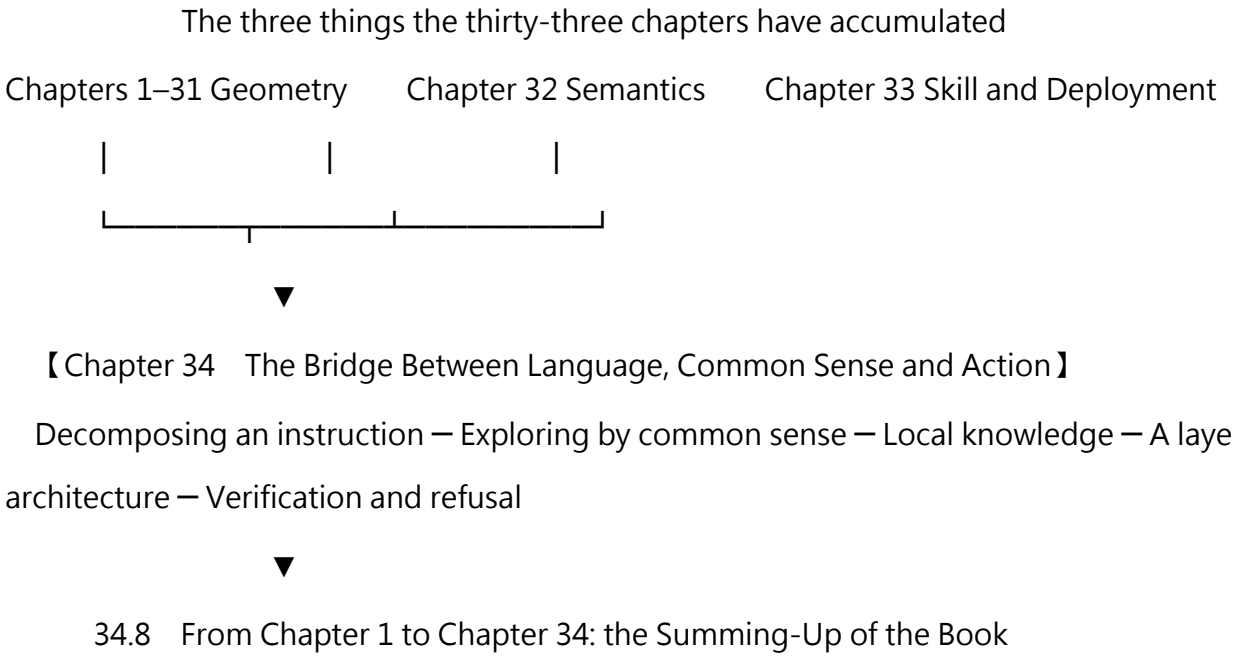
This is the last chapter of the book, and the one that gathers up the thirty-three before it.

Up to now we have three things: Chapters 1 to 31 gave the robot an accurate geometric map, so that it knows where I am and what shape the environment has; Chapter 32 gave the map a meaning, so that it knows that is a door and that is a person; and Chapter 33 let it learn a skill in simulation and sent the model into the computing module on its body.

One thing is still missing: a purpose. When the user says go to the meeting room and turn off the projector, the robot must turn one sentence into a succession of executable subgoals, must reason on a floor it has never visited about which side the meeting room is likely to be on, and must know where to go on looking when it cannot find the goal. This is the subject of this chapter—the bridge between language, common sense and action.

This chapter carries the summing-up of the whole book as well. Section 34.8 will use one complete task to put every part from Chapter 1 to Chapter 34 back where it belongs, and explain why not one of the nine parts could be spared.

Figure 34-0 The Link Map of Chapter 34



Learning Objectives

This chapter begins from how one sentence becomes a coordinate and takes up in turn the common-sense prior, the local knowledge base, the layered architecture and the mechanism of verification, then examines honestly the real level of today's vision-language models in navigation, and closes with the summing-up of the book. It has three figures computed from experiments, each answering one question: what common sense is worth, what happens when common sense is wrong, and why there must be a layer of verification. By the end of this chapter students should be able to:

1. Explain the stages by which a natural language instruction becomes a navigation goal, and point out the mode of failure possible at each.
2. Explain how a common-sense prior shortens the cost of searching in object-goal navigation, and quantify what it is worth.
3. Explain what a prior pointing the wrong way costs, and judge from that when

the semantic level should not be trusted.

4. Design a local spatial knowledge base, and explain three advantages of retrieval-augmented generation over fine-tuning a model.
5. Explain a system architecture layered by time scale, and point out the update rate and the responsibility of each layer.
6. Estimate the overall success of a multi-step task by a model of multiplied probabilities, and explain why verification and retrying improve it so greatly.
7. Explain how far today's vision-language models stand from human performance in embodied navigation, and read the results of the relevant benchmarks.
8. Map the contents of the book's nine parts onto the stages of one complete task.

Core Concepts

Term	Meaning	Related Sections
Embodied intelligence	The intelligence that arises as an agent interacts with its environment through a body	34.1, Chapter 33
Language navigation	Navigating an unknown environment by an instruction in natural language	34.2, 34.6
Object-goal navigation	Given the name of an object, finding where it is	34.3

Common-sense prior	The statistical knowledge that an object of some kind is usually in a space of some kind	34.3
Semantic exploration	Deciding the next direction of exploration by a semantic prior	34.3, Chapter 16
Retrieval-augmented generation	Retrieving the relevant information from a knowledge base before inferring	34.4
Scene graph	A structured representation describing a scene by nodes and relations	34.1, Chapter 32
Layered architecture	Dividing a system into layers by time scale, each with its own office	34.5, Chapter 3
Mechanism of refusal	A lower layer may overrule an upper one's decision to keep things safe	34.5, 34.7
Vision-language-action model	A model that outputs actions directly from images and language	34.6, Chapter 33

Table 34-1 The Core Terms of This Chapter

34.1 From Knowing What a Thing Is to Knowing What to Do

At the close of Chapter 32 the robot had a map marked with semantics. This section explains what is still missing between that and being able to execute one sentence.

One: the Three Levels, Once More

Chapter 32 described robot intelligence in three levels: the geometric level answers where am I and what shape the environment has, the semantic level answers what have I seen, and the task level answers what should I do next. The first two this book has dealt with; this chapter deals with the third.

The task level differs from the first two in kind: the input of the first two is a sensor and the output a description of the world; the input of the task level is a person's intention and the output an action. A description may be made accurate, but an intention is often vague—turn off the light in the meeting room says neither which meeting room nor where the switch is. The better half of the task level's work is supplying what was not said.

Two: How Many Assumptions Hide in One Sentence

Take go to the meeting room and turn off the projector: the information the robot must supply for itself includes at least these.

- Which meeting room? (If there are several, it may have to be inferred from where the user is or from the calendar.)
- Where is the meeting room? (Local spatial knowledge is needed, Section 34.4.)
- What does a projector look like? (The recognition of the semantic level is needed, Chapter 32.)
- Where in the meeting room is the projector? (Common sense is needed: usually on the ceiling or on the lectern.)

- How is it to be turned off? (The remote? A button on the body? Or an infrared signal?)
- How is it confirmed that it really went off? (Verification is needed, Section 34.5.)
- What if it cannot be turned off? (Degradation and reporting are needed, Section 34.7.)

Of these seven, only the third was dealt with in Chapter 32. The other six need local knowledge, common-sense reasoning, knowledge of actions and a mechanism of verification—which is exactly what this chapter is to supply.

Three: Why a Scene Graph Is More Useful than a Semantic Map

Chapter 32 mentioned the scene graph: recording not merely that there is a chair here but the structure of relations—the chair is beside the table, the table is in the meeting room. At the task level that difference becomes crucial.

The reason is that spatial relations in language are relative. The cup on the table, the fire extinguisher beside the door, the third seat in the row by the window—none of these can be expressed on a flat semantic map, which records only that there is an object of some class at some coordinate. To answer the cup on the table, the system must know which cup's relation of support points to which table.

A Distinction: the Three Forms a Navigation Goal Takes

A point goal: a coordinate given directly (go to (12.4, 8.1)). This is the form the first thirty-one chapters deal with, and the form every upper layer must finally convert into.

An object goal: a category of object given (go and find a water cooler). The

position of the goal is unknown and must be searched for—the subject of Section 34.3.

An instruction goal: a passage of natural language given (go to the end of the corridor, turn right, and go into the second classroom). It must be understood and executed step by step—what Section 34.6 assesses, and the hardest of the three at present.

34.2 How a Language Instruction Becomes a Navigation Goal

This section takes that chain of conversion apart, because every link in it is a possible point of failure.

One: Five Stages

Stage	Input	Output	Typical failure
① Parsing the intention	Natural language	A structured description of the task	An unclear reference, an ambiguous verb
② Locating the place	The description of the task	A region on the semantic map	The name of the place is not in the knowledge base
③ Locating the goal	The region + the object category	A three-dimensional coordinate	The object is not where it was, or is recognized wrongly
④ Planning the path	A three-dimensional coordinate	A feasible path	The goal is unreachable, the door is shut

⑤ Executing and confirming	The path	Completion or a report	It arrived and did not confirm; it was interrupted on the way

Table 34-2 The Five Stages from Language to Action

It is worth noting that everything after stage ④ has already been dealt with entirely by the first thirty-one chapters of this book. What this chapter and Chapter 32 add are stages ① to ③. Which is to say that language navigation does not replace SLAM but attaches a stretch of conversion in front of it.

Two: The Output Should Be Structured, Not Free Text

An important principle in implementation: what an upper layer sends a lower one should be structured data (goal coordinates, a list of objects, an action code) and not free text.

The reason is practical. Free text cannot be parsed reliably, nor verified. If the language model answers walk toward that room ahead on the left that looks like a tea room, how is the downstream system to turn it into a coordinate? Whereas if it answers goal region: tea room, confidence 0.72; candidate coordinate (23.3, 13.9), the downstream can query the map and verify whether that coordinate is reachable, whether it lies in a forbidden zone, and whether it conflicts with the cost map.

This too is why having a language model output velocity commands directly is poor engineering design. It skips every intermediate representation that could be verified, and turns a system that can be checked into a black box that cannot.

Three: Uncertainty Must Be Passed Down

Chapter 11 spent a whole chapter on probability and uncertainty, and the idea applies just as well at the task level, where it is often overlooked. A language model will usually say the meeting room is on the east side of the third floor in the same confident tone whether it looked the fact up in a knowledge base or guessed it.

The practical course is to have every goal the upper layer outputs carry a confidence and a mark of its source (looked up in the knowledge base / inferred from common sense / a pure guess). The lower layer decides from that mark how careful to be—a goal from the knowledge base may be driven to directly, while a goal that is a pure guess should have a cheap exploratory trip sent out first.

34.3 The Common-Sense Prior and Semantic Exploration

A washbasin is usually in a toilet or a tea room—knowledge of this kind cannot be derived from a geometric map, and yet it changes a robot's efficiency of search a great deal. This section quantifies what it is worth with an experiment that can be reproduced in full.

One: the Experimental Setting

The scene is one floor of a departmental building: a central corridor with twelve rooms on either side, of six kinds (classroom, laboratory, office, tea room, meeting room, toilet, storeroom). The robot sets out from the entrance to find a specified object (a water cooler, a fire extinguisher, a projector, a first-aid box, a ladder, or a copier).

The crucial part of the setting is that both strategies are given the same map. This is not a comparison of having a map against having none—that would not be fair—but a comparison of the order of search under one and the same prior map:

- Geometric order: go each time to the nearest room not yet searched. This is the baseline, using no semantic information at all.
- Semantic order: sort by the probability that an object of this kind appears in a room of this kind, from high to low, taking the nearer room where the probabilities are equal.

The objects are placed at random by a set of real conditional probability distributions (a water cooler, say, is in the tea room with probability 55 percent, in the corridor 30 percent, and in a meeting room 15). On entering a room the robot sweeps back and forth (the length of the sweep estimated by the coverage planning of Chapter 16), and the object counts as found when it comes within a sensing range of 3.5 meters with the line of sight not blocked by a wall. Six hundred random trials are run in all.

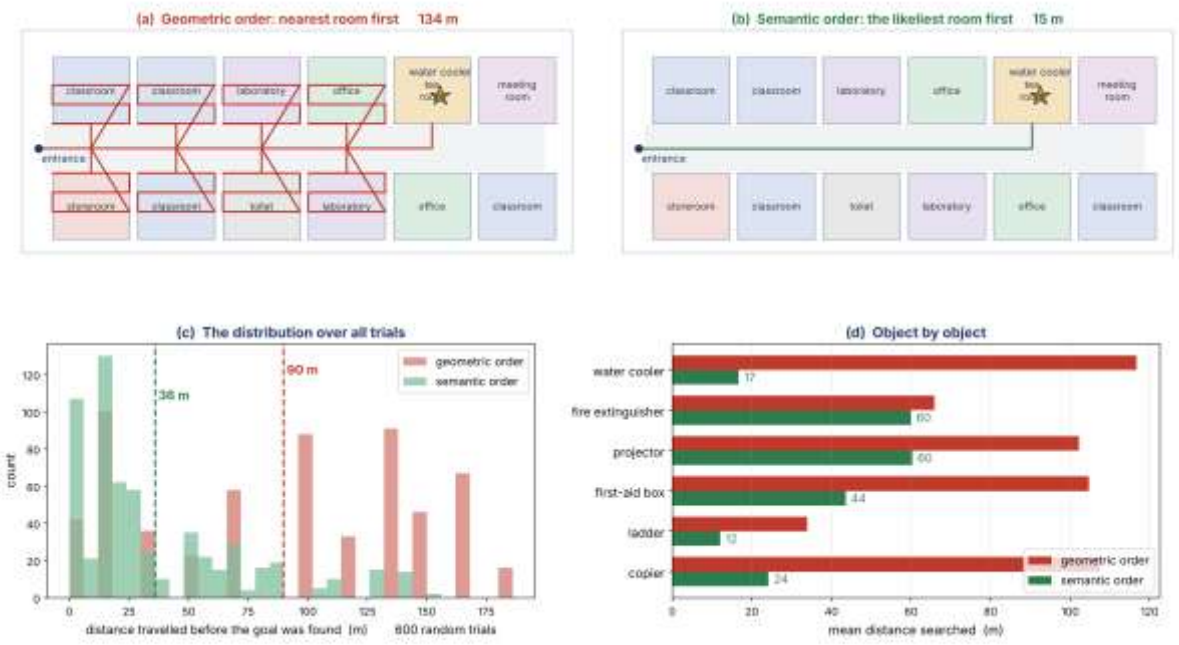


Figure 34-1 The Same Map, Two Orders of Search

Panels (a) and (b) draw the same trial: the water cooler is in the tea room. The geometric order begins from the classroom nearest the entrance and presses eastward room by room, travelling 134 meters before reaching the tea room. The semantic order goes

straight to the tea room and travels only 15. What is drawn is the path of movement between rooms; the sweeping within a room is not drawn, though its distance is counted.

Panel (c) is the overall distribution over 600 trials. The mean of the geometric order is 90 meters and of the semantic order 36, a mean shortening of 60.1 percent; by the median it is 100.9 meters against 18.3, a shortening of 81.9. The two distributions differ in shape as well: the geometric order shows distinct peaks, each corresponding to which room in the order of search the object fell in, while the semantic order is heavily concentrated within 30 meters.

Panel (d) takes the objects one by one, and the benefit differs greatly. The ladder falls from 34 meters to 12 and the copier from 108 to 24, because their distributions are highly concentrated (the ladder is in the storeroom 70 percent of the time, the copier in an office 65). The fire extinguisher falls only from 66 meters to 60, because it is in the corridor with probability 60 percent—and the corridor is where the robot passes in any case, so a semantic prior is of little help there.

That difference is itself a useful conclusion: the value of a common-sense prior depends on how concentrated the object's distribution is. For objects that are everywhere (fire extinguishers, waste bins, hydrants) the return on effort spent building semantic reasoning is low; for objects that are only in particular places (a ladder, a copier, a particular instrument) the return is high.

A Number Often Skipped Over

In these 600 trials the semantic order travelled farther than the geometric one

in 15.3 percent of cases.

The reason is that the room of highest probability is not necessarily the room the object is really in this time. When the water cooler falls in the meeting room on its 15 percent, the robot that rushed off to the tea room has made a wasted trip.

Which is a reminder that a semantic prior improves the expectation and not every single run. In assessing any semantic navigation system, looking only at the mean underestimates its variation—a principle of assessment stressed over and over in Chapter 31.

Two: What Happens When the Prior Is Wrong

The experiment above assumes that the robot's common sense is right. But a vision-language model's common sense is learned from data on the internet and need not match your building. The copier is in an office holds in most places, but what if this building puts all its copiers in an open area of the corridor?

The experiment below sweeps the accuracy α of the prior: with probability α the true distribution is used, and otherwise the ordering of room types is shuffled at random. One extreme case is added as well—a prior that points the wrong way, so that the higher the true probability of a room the later it comes in the order.

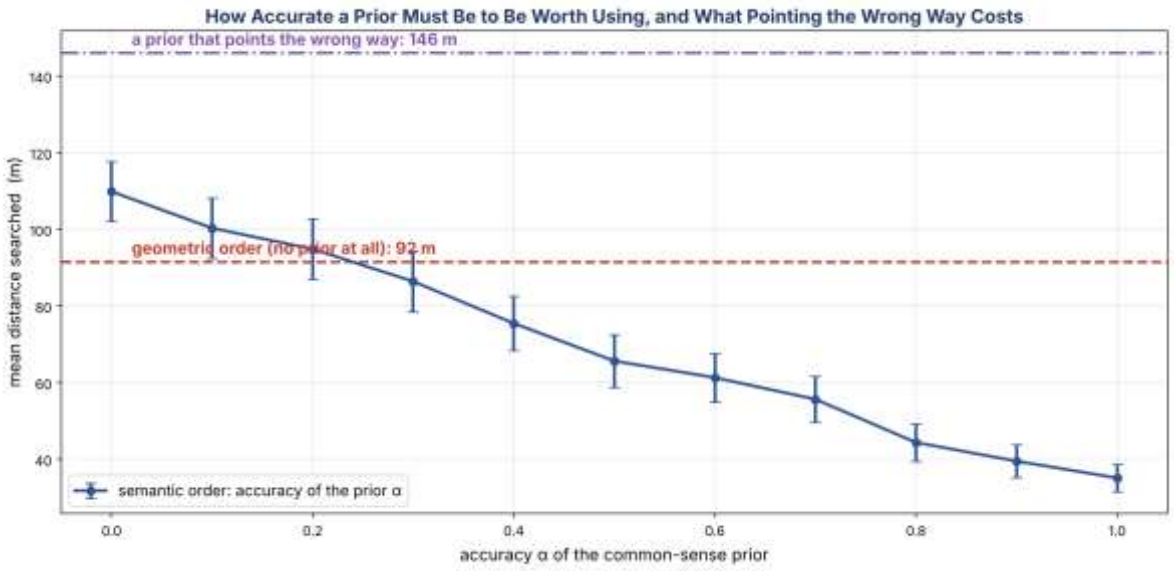


Figure 34-2 How Accurate a Prior Must Be to Be Worth Using, and What Pointing the Wrong Way Costs

The blue line is the semantic order as α varies, with 95 percent confidence intervals for error bars. It falls from 110 meters at α = 0 all the way to 35 at α = 1, roughly linearly. That linear relation is itself good news: a common-sense prior need not be perfect to be useful, and every tenth of accuracy gained saves about seven meters.

The red dashed line is the geometric order with no prior at all (92 meters). Note where the blue line crosses it: at about α = 0.25. Below that the prior is worse than useless—a wholly random order of search (110 meters at α = 0) is in fact somewhat worse than one that goes from near to far, since going to the nearer room first at least saves travel at the start, while a random order does not even do that.

The real risk is in the purple dotted line: when the prior points the wrong way the mean distance of search is 146 meters, 60 percent worse than using no prior at all. This is the most dangerous mode of failure of the semantic level—it does not merely fail to help but actively leads the robot the wrong way, and does so in the same confident tone.

The lesson for system design is clear: the semantic level's suggestions must be able to

be overruled. A reasonable course is to set a budget—if the first two suggestions of the semantic order have been searched without finding the object, fall back on the geometric order and trust the prior no further. This is exactly the concrete form of the mechanism of refusal of Section 34.5.

34.4 RAG: Letting a Robot Read the Spatial Knowledge of Its Own Site

The common-sense prior of the last section deals with knowledge in general. This section deals with the other half: the knowledge peculiar to this one building.

One: the Problem and the Solution

Here is a practical problem: however clever a vision-language model may be, it does not know which building and which floor this school's registrar's office is on—that is the knowledge of a particular institution, and it is not in the model's training data.

The solution is retrieval-augmented generation (RAG): build a local spatial knowledge base, and when the robot receives an instruction, retrieve the relevant information from the knowledge base first and then hand it to the model to reason over. The model is responsible for understanding and reasoning in language, and the knowledge base for the facts.

Figure 34-3 An Example Structure for a Campus Spatial Knowledge Base

【The topology of the floors】

First floor: the gate → the hall → the registrar's office → the student affairs office → the

stairs → the lift

Second floor: the lift → the corridor → the computer classroom → the meeting room

【The attributes of a place】

The registrar's office: first floor, east of the hall, open 08:00–17:00, the door an automatic glass one

The intelligent robotics classroom: third floor, entrance → work benches → the robot dog test area → the LiDAR mapping area → the charging area

【The rules of passage】

The robot may not enter: the inside of a laboratory, the stairs (except a stair-climbing platform)

Regions in which it must slow: the hall, the doorway of a classroom, a corner

【What common queries correspond to】

"office" → the registrar's office, the student affairs office "charging" → the charging stations on each floor

Two: the Three Advantages of RAG

1. It can be maintained: when the layout of the campus changes, only the knowledge base need be updated and the model need not be retrained. A classroom changes its use and one line of text is changed.
2. It can be verified: the robot's answer can be traced to the entry in the knowledge base it came from, which makes debugging and accountability easier. This bears directly on the question of explainability in Section 34.7.
3. It is cheap: the cost of building a knowledge base is far lower than fine-tuning a large model, and it suits a teaching project better—one structured text file is

enough to begin.

Three: Three Practical Examples of a Query

- How do I get from the entrance to the robot dog test area?—RAG retrieves the internal topology of the intelligent robotics classroom, the model generates a description of the route from it, and that is turned into a sequence of navigation goals.
- If there is a student sitting on the floor ahead, how should the robot go around?—real-time perception (a person detected) is combined with the rules of passage in the knowledge base (slow down, give way) to produce a strategy for going around.
- How should a robot vacuum plan to cover a whole classroom?—the map (Chapter 13) is combined with coverage planning (Chapter 16), a typical example of the semantic and geometric levels working together.

Four: A Practical Recommendation on Architecture

For a project implementation the division of labor advised is this: a structured text file or a simple database will serve for the RAG knowledge base (there is no need to start with a vector database); the language model is responsible for understanding language and judging what it sees; and every decision bearing on safety (obstacle avoidance, speed limits) is still guarded by the classical geometric system. This is the principle the next section stresses repeatedly.

> **The Division of Labor Between the Knowledge Base and the Common-Sense Prior**
>
> A point easily confused: the common-sense prior of Section 34.3 and the

knowledge base of this section are both knowledge outside the model, and the two are wholly different in character.

The common-sense prior is statistical and general across sites (a ladder is usually in the storeroom), and answers where to look first. The knowledge base is definite and particular to a site (the storeroom is on the west side of the first floor), and answers where that place is.

A good system wants both: look up with the knowledge base all that can be looked up, and leave only what cannot to the common-sense prior to guess—remembering to mark it a guess (Section 34.2).

34.5 The Layered Architecture and the Mechanism of Verification

This section explains how to assemble the components of the sections above into a system that does not run out of control when an upper layer goes wrong.

One: Layering by Time Scale

Figure 34-4 The Layered Architecture of a Language Navigation System

【The task layer】a second and above the language model agent

- Understand the natural language instruction, query the knowledge base (Sections 34.2, 34.4)
- Decompose the task, choose the direction of exploration, judge an anomaly (Section

34.3)

↓ output: a semantic goal (where to go, what to find) + a confidence + a source

【The semantic layer】0.1–1 second the AI vision models

- Object detection, segmentation, depth estimation (Chapter 32)
- Building and querying the semantic map

↓ output: geometric goal coordinates, labels of dynamic objects

【The geometric layer】10–30 Hz SLAM + navigation (Chapters 1–31)

- Localization and mapping, path planning, the cost map

↓ output: velocity commands

【The control layer】100 Hz and above the chassis / flight controller (Chapter 33)

- Motion control, immediate obstacle avoidance, the safe stop

【The principle of safety】an upper layer may go wrong; a lower layer must hold the line of safety

This layering, the hybrid architecture of Chapter 3, the two subsystems of Cartographer in Chapter 15, the two-layer design of LOAM in Chapter 18, and the degradation architecture of the drone in Chapter 26 are one and the same pattern of system design carried into the age of AI—layer by time scale, and let each layer do its own office. By this point students should feel deeply that a good principle of architecture holds good across eras.

Two: Four Practical Points in Integration

1. Design it asynchronously: AI inference takes time and must never block the navigation loop. It should be called asynchronously, and the robot kept in a safe behavior while it waits (continuing with the previous goal, or stopping).
2. An explicit interface: what the AI layer outputs should be structured data and

not free text (Section 34.2). This lets the downstream system parse and verify it reliably.

3. A mechanism of refusal: the geometric layer should be able to overrule the AI's decision. If the AI says go forward while the cost map shows an obstacle ahead, the geometric layer must win. The purple dotted line of Figure 34-2 shows what the want of such a mechanism costs.

4. A path of degradation: when the AI service is unavailable (disconnected, timed out, the model crashed), the system should fall back on purely geometric navigation rather than halt—exactly the principle of degrading failure by degrees of Section 26.2.

Three: Why a Multi-Step Task Must Have Verification

The mechanism of refusal above sounds like an insurance for safety. But it has a more fundamental use as well: without it a multi-step task cannot have a usable rate of success at all. A simple model below explains why.

Suppose a task has n steps, each succeeding with probability p, and the steps are independent. Then the overall rate of success is p to the nth power—this is Formula One. The destructiveness of that product is far greater than intuition suggests.

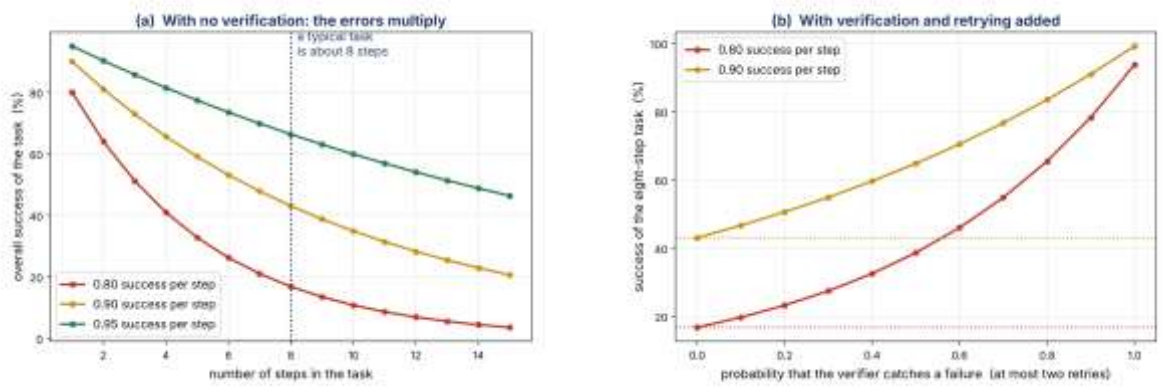


Figure 34-5 The Success of a Multi-Step Task, and the Effect of Verification and

Retrying

Panel (a) draws the consequence of the product. For a task of eight steps (the five stages of Section 34.2 plus opening the door, lining up, and reporting—eight is a conservative estimate), even with a 90 percent rate of success at every step the overall rate is only 43.0 percent; at 80 percent per step only 16.8 remains. For an eight-step task to reach 66.3 percent, every step must manage 95.

This explains a common puzzlement: why does every subsystem of a robot look all right while the whole thing keeps failing? It is not that any one link is particularly bad; the product itself is merciless. It also explains why dividing the task more finely is not necessarily a good thing—the finer the division, the more the steps, and the heavier the penalty of the product.

Panel (b) is the remedy. A verification is added after every step: if the verifier judges the step to have failed, it is retried (at most twice). The horizontal axis is the probability that the verifier catches a failure, and the dotted line is the level with no verification. Taking a system at 80 percent per step: the overall rate of success rises from 16.8 percent to 38.7 at a detection rate of 0.5 and to 93.8 at a detection rate of 1.0. A system at 90 percent per step rises from 43.0 to 99.2.

Note the size of that improvement: a perfect verifier turns a system that fails eight times in ten into one that succeeds nine times in ten, with the ability of every underlying step entirely unchanged. This is one of the most important engineering conclusions of the chapter—for a multi-step task, investing in knowing that one has got it wrong pays far better than investing in doing it better.

And verification is usually much easier than execution. Driving to the door of the

meeting room needs a complete navigation stack, while confirming whether one is at the door of the meeting room may need only one reading of the room number, or one comparison of the current pose with the goal pose. That asymmetry—verification being simpler than execution—is the fundamental reason a layered architecture can stand.

Four: Formula One—the Success of a Multi-Step Task

Let the task have n steps, the ith succeeding with probability p_i, the steps independent, with no verification and no retrying. Then:

$$P_{task} = p_1 \times p_2 \times \dots \times p_n$$

When every step succeeds with the same probability this simplifies to $P = p^n$. Adding verification (with detection rate r) and at most k retries, the effective rate of success of a single step rises to:

$$p_{eff} = p + (1 - p) \cdot \Sigma_{j=1..k} (r (1 - p))^{j - 1} \cdot r \cdot p$$

where r·p is the probability of being caught and retried successfully. When r → 1 and k is large enough, p_eff → 1 and the overall rate of success approaches 1; when r → 0, p_eff → p and we return to the case of panel (a). The curves of Figure 34-5(b) are the numerical results of this formula at n = 8 and k = 2.

34.6 Where Things Stand: How Good Vision-Language Models Really Are at Navigation

The five sections above described an ideal architecture. This section examines honestly how well the topmost layer of that architecture does at the level of the technology of 2025 and 2026.

One: A Benchmark for Embodied Navigation

A benchmark for embodied navigation published at the end of 2025 gives a fairly clear answer. Its design is this: the model is given a photograph of a real scene, an instruction in text, and a type of embodiment (a person, a quadruped robot, a wheeled robot, a bicycle), and is to draw on the image the trajectory it ought to travel. The dataset holds 1,000 situations and more than 3,000 expert trajectories, and the scoring combines the dynamic time warping distance of the trajectory, the endpoint error, and a penalty computed from semantic segmentation for the places this embodiment ought not to go.

Eight contemporary vision-language models were assessed. The results are as follows:

Method	Score
A human expert	75.40
Predicting the path alone (told directly where the goal is)	56.55
The straight-line baseline to a known goal	51.89
Complete trajectory prediction (the best model)	34.38
Predicting the goal point alone	29.65

Table 34-3 The Results of the Embodied Navigation Benchmark (a higher score is better)

Two: Three Numbers Worth Remembering

First, the gap between the best model's 34.38 points and the human expert's 75.40 is 41.02 points. This is not a gap to be described as caught up with in another six months. The vision-language models of 2026 are still far from human at embodied navigation.

Second, and the most worth savoring: a straight line to a known goal scores 51.89 points, far above the 34.38 of the best model's complete trajectory. A straight line that looks at nothing in the environment, avoids no obstacle and goes around nothing beats the complete planning of the strongest model of the day. Which shows that the model loses its marks not chiefly on the shape of the path but on having no clear idea where the goal is.

Third, which bears that judgment out: when the goal is given directly and the same model predicts the path alone, it scores 56.55; the other way round, predicting the goal point alone, it drops to 29.65. Which is to say that the model can walk and cannot recognize—mapping the goal named in the language onto the right place in the image is the real bottleneck at present.

Three: What This Means for System Design

These three numbers support the design principles of Sections 34.4 and 34.5 directly. Since the model's weakest link is locating the goal, the system should so far as possible not let it do that at all—a place that can be looked up in the knowledge base is never left for the model to guess (Section 34.4); where it must guess, the guess is marked as such and a budget for searching is set (Sections 34.2, 34.3); and the cost of a wrong guess is absorbed by the mechanisms of verification and refusal (Section 34.5).

Conversely, what the model is relatively good at should be left to it: understanding language, reading an image, generating a report, describing a route when the goal is known. This is a practical division of labor—the question is not whether a language model can do navigation but which parts of navigation suit being left to it.

Another Fact of the Present: Real Deployment Is Still Rare

A survey of 2026 points out that progress in language navigation research is mostly still confined to real-to-sim settings at the level of software, with relatively little research on sim-to-real and real-world deployment, and that practical schemes for edge deployment are quite scarce as well.

This echoes the conclusion of Chapter 33: a result in simulation and performance on a real machine are two different things. For students this is in fact an opportunity—a language navigation system that really runs on a real campus, however simple in function, has substantial research value at present.

34.7 Risks, Limits and the Road Ahead

Before closing, three things must be discussed: the risks, the limits, and the direction.

One: Four Great Risks

1. Hallucination: AI may see things that are not there or reason wrongly, and it usually does not express uncertainty. A robot that trusts AI entirely may hit a wall or take the wrong road. The purple dotted line of Figure 34-2 is the quantified form of this very risk. The remedy is to keep the geometric layer's right of verification and refusal (Section 34.5).
2. The systematic character of a misjudgment: an AI's errors are often not random but systematically biased—always mistaking a certain class of object under a certain light, say. This is more dangerous than random error, because it keeps happening and retrying cannot save it (the model of verification in Section 34.5 assumes retries to be independent, and a systematic error breaks that assumption). The remedy is thorough testing on the site and analysis of the

errors (Chapter 32, Section 33.5).

3. Latency: inference with a large model takes time, and putting it inside a real-time control loop is a disaster. The remedy is a strict layered architecture—AI makes only decisions on the scale of a second and above (Section 34.5).
4. Security: which includes information security (the model or the knowledge base being tampered with), privacy (the images a robot takes may contain personal information), and accountability (whether the grounds of a decision can be traced when something goes wrong). These are not only technical questions but questions of ethics and regulation.

Two: Four Present Limits

- The demand for data: training or fine-tuning a model needs labeled data, and labeling is expensive. This is the chief obstacle for a project or a small team—and the reason distillation, few-shot fine-tuning and synthetic data (Sections 33.5 and 33.8) are regarded so highly.
- The gap in generalization: a model does well in the laboratory and falls off at another site. The discussion of the domain gap in Section 33.5 applies to the semantic level just as well.
- Explainability: why did the AI judge so? When it goes wrong, diagnosis is often difficult. This stands in sharp contrast to classical geometric methods, where every step can be traced, and is one of the reasons Section 34.4 argues for a knowledge base rather than fine-tuning.
- Computation and energy: as Section 33.8 described, even after quantization and distillation AI remains one of the most resource-hungry components on a robot.

Three: Four Directions for the Future

1. A deep integration of SLAM with foundation models: no longer SLAM plus an AI module, but one architecture in which a foundation model handles perception, mapping and planning together.
2. World models: letting a robot understand not only what is now but predict what will happen next. This matters greatly for planning in a dynamic environment, and connects directly with the training in simulation of Chapter 33—a good enough world model is itself a simulator one can train inside.
3. Robot foundation models: a general agent across platforms—one and the same model deployable on wheeled, quadruped, humanoid and flying platforms alike. This would lower the threshold of robot development greatly.
4. Lifelong learning: a robot that goes on learning and adapting after deployment, rather than staying at the ability it left the factory with. The on-device self-supervised learning mentioned in Section 25.6 is an early attempt in this direction, and the map maintenance of Section 19.6 its most practical rudiment.

34.8 From Chapter 1 to Chapter 34: the Complete Journey of One Robot

As the last section of the book, let us use one complete task to string the thirty-four chapters together.

One: One Task, Nine Parts of Knowledge

The situation: a service robot at a university of technology in southern Taiwan receives an instruction—please go to the intelligent robotics classroom on the third floor, check the state of the equipment in the charging area, and report anything amiss.

1. Understand the instruction: the model parses out the goal place and the content of the task (Section 34.2). The output is a structured description of the task, not free text.
2. Query the knowledge base: RAG retrieves that the intelligent robotics classroom is on the third floor, and that its internal topology is entrance → work benches → robot dog test area → LiDAR mapping area → charging area (Section 34.4). This step saves the robot from guessing.
3. Plan the route: plan a path on the existing map from the current position to the third floor, the lift included (Chapters 3 and 29).
4. Move and localize: LiDAR SLAM or visual-inertial SLAM keeps estimating the pose (Chapters 13 to 23), localizing within the existing map (Chapters 11 and 29), with the IMU holding things up through turns and vibration (Chapter 7).
5. Pass the lift lobby: seeing an artificial marker, obtain an absolute pose correction and confirm the floor (Chapters 4 and 24).
6. Avoid a pedestrian: on detecting a person, remove his features so that they do not disturb the localization, and at the same time trigger slowing and giving way (Chapters 24 and 32).
7. Reach the charging area: find the goal by the semantic map and complete the precise alignment by visual servoing (Chapter 24). If it is not to be found on the semantic map, decide where to look by the common-sense prior, and fall back on a geometric search after two attempts (Section 34.3).
8. Inspect and judge: photograph the equipment, have the AI judge whether anything is amiss, and mark the finding with a three-dimensional coordinate (Chapters 31 and 32). Every skill of movement and manipulation comes from the

training in simulation of Chapter 33.

9. Verify every step: confirm the result after each stage is finished, and retry on failure (Section 34.5). Without this step the overall success of the eight steps above would be too low to use.
10. Report: generate a report in natural language, with the position and the photographic evidence attached (Section 34.2).

Every part of this book contributes to that one errand: the ideas of Part I, the sensors of Part II, the mathematics of Part III, the SLAM systems of Parts IV to VI, the edge computing of Part VII, the implementation tools of Part VIII, and the AI semantics, training in simulation and language navigation of Part IX.

And the crucial thing is this: not one of the nine parts could be spared. Without the geometric level the robot hits a wall; without the semantic level it does not understand the instruction; without training in simulation it never learns to move; without the toolchain nothing runs at all. This is the message the book has meant to convey from its first page to its last—SLAM is not a single technique but a whole body of knowledge whose parts hold one another up.

Two: One Thread Running Through the Book

The book began at Chapter 1 with a robot that has no map and does not know where it is, and has passed through sensors, mathematics, 2D and 3D systems, visual methods, miniature and swarm robots, and a complete implementation, to arrive at last at AI semantic navigation, training in simulation and the understanding of language.

If one thread is to be said to run through these thirty-four chapters, it is this: only by understanding a limit can a limit be broken.

We have discussed over and over the weakness of every sensor (Part II), the mode of failure of every algorithm (Parts IV to VI), the constraint on resources of every platform (Part VII), the order of debugging of every system (Part VIII), and the source of the gap in every model (Part IX). This is not pessimism; it is that the nature of engineering is to find a solution within limits.

The arrival of AI has not changed that nature. A language model lets a robot understand and think things through, while walking steadily and not running into people still rests on the geometry and control of the thirty-one chapters before. The numbers of Section 34.6 say it plainly: at embodied navigation the strongest model of the day is still far from human, and a straight line to a known goal even beats its complete planning. Neither can be spared—and so it will be for the foreseeable future.

Three: A Few Last Words to the Student

The two settings of this book—a university's intelligent robotics program and a high school's AI and robotics club course—stand for two stages of learning, and share one conviction: SLAM is not some far-off frontier technology. Its core version will run on a processor of the grade of a mobile phone, its principles can be worked out with pen and paper, and its fruits are turning quietly in the robot vacuum in your own living room.

Every experimental figure in this book was computed on an ordinary computer. The loop closure of Chapter 15, the registration of Chapter 17, the motion compensation of Chapter 18, the inspection route of Chapter 19, the parallel simulation and parameter identification of Chapter 33, and the semantic search of this chapter—not one of them needed expensive equipment. What was needed was only to state the problem clearly, do the experiment cleanly, and look at the numbers honestly.

May students take the foundation this book has laid and go on to build robots of their own—whether a robot dog inspecting a campus, a drone scanning a historic site, or a little car that can only follow a line. Behind every robot that finds its own way is the same body of understanding that took humanity forty years to build.

Now it is your turn.

Cases

Case 34-1 One Sentence, Eight Steps: Why Verification Matters More than Ability

This case puts the numbers of Section 34.5 back into a concrete situation.

Suppose a student has built a campus delivery robot, whose task is to take this document to the registrar's office on the third floor. Taken apart, that is eight steps: understand the instruction, look up the place, plan the route, take the lift, walk to the door, recognize the room number, enter, hand over and confirm.

Suppose every step manages a success rate of 80 percent—which does not sound bad, every subsystem succeeding eight times in ten. But the overall rate is 0.8 to the eighth power, which is 16.8 percent. Eight errands in ten will go wrong somewhere.

A team's instinct is usually to improve the weakest link. Suppose a whole semester is spent raising every step from 80 percent to 90—a considerable engineering effort. And the result? The overall rate rises from 16.8 percent to 43.0. A large improvement, and still nearly six failures in ten.

The other road is to change nothing in any step but add a check after each. Was the lift button pressed? Compare the floor sensor. Are we at the door? Compare the current

pose with the goal pose. Is the room number right? Take a photograph and read the text. If the check fails, do that step again, at most twice.

Computed by the model of Section 34.5: if the check catches half the failures, the overall rate rises from 16.8 percent to 38.7; if the check is nearly perfect, to 93.8. And the ability of every underlying step has not changed at all.

What this case is meant to convey is more than a trick with numbers. It is a judgment about the order of engineering priorities: when a multi-step system performs poorly as a whole, asking whether it knows it has got something wrong is usually more effective than asking how to make it do better. This is the same line of thought as the order of debugging in Chapter 31 and the degradation design of Chapter 26.

Case 34-2 When the Robot's Common Sense Points the Wrong Way

This case takes up a mode of failure students are quite likely to meet in a project and are seldom warned of beforehand.

The situation: a student adds semantic search to an inspection robot. To find a copier, the robot first asks the language model where a copier usually is, receives the answer an office, and so searches every office first. In the building they tested in, the strategy did excellently—the distance searched fell by seven tenths.

Then they moved the system to another building. To save power and simplify management, that building had put all its copiers together in the open area beside the lift on each floor, and there was not one in any office. The result was that the robot went through every office first, every time, and only then found the goal in the corridor it thought least likely.

Figure 34-2 quantifies the situation. When the prior points exactly the wrong way the mean distance of search is 138 meters, against the 90 of a geometric order using no prior at all—the semantic level has set the system back by 54 percent. Worse, the system will not know it is wrong: every time it reasonably goes to the offices first, and every time it reasonably fails.

The middle stretch is worth noting. The blue line of Figure 34-2 stands at 84 meters at $\alpha = 0$ (a wholly random prior), slightly better than the geometric order's 90. Which is to say that an ignorant prior is not to be feared; what is to be feared is a confident and wrong one. And that is precisely the kind a language model is most apt to produce—it seldom says I do not know.

The remedies come at three levels. The least is to set a budget for searching: search the first two places the semantics suggest, and fall back on the geometric order if they fail. Further is to write local observation back into the knowledge base: once a copier has been found in the corridor, that fact should be recorded rather than guessed at afresh from general common sense next time (Section 34.4). The most thorough is to let the prior itself be learned—which is exactly the problem the lifelong learning direction of Chapter 33 is to address.

Case 34-3 A Straight Line Beats the Strongest Model

This is a case fit for the end of the book, because it states at once the value of AI and the boundary of it.

A benchmark for embodied navigation at the end of 2025 asked models to draw on a photograph of a real scene the trajectory a robot ought to travel. Eight of the strongest contemporary vision-language models were assessed, and the scoring weighed the

shape of the trajectory, the accuracy of the endpoint, and a penalty for the places this embodiment ought not to go.

The most surprising item in the results is this: a straight line to a known goal position scored 51.89 points, while the best model's complete trajectory prediction scored only 34.38. A straight line that avoids no obstacle, goes around nothing, and looks at the environment not at all beat the complete planning of the strongest model of the day.

The first reaction may be to laugh at the model. But thought over, the result says not that the model is stupid but that the great bulk of the score comes from the position of the goal. A further experiment confirms it: the same model, told the goal position directly and predicting only the path, leaps to 56.55; the other way round, predicting the goal point alone, it drops to 29.65.

Put otherwise, the model can walk and cannot recognize. It understands a spatial description such as go forward along the corridor and avoid that pile of boxes, and cannot reliably judge which door in the picture is the meeting room the user spoke of.

This diagnosis is of great value to system design, because it says where the engineering effort should go. Since locating the goal is the bottleneck: what can be looked up in the knowledge base is not guessed at (Section 34.4); where an artificial marker will serve, vision alone is not relied on (Chapters 4 and 24); and where it must be guessed, the confidence is marked and a budget set (Sections 34.2 and 34.3).

It says at the same time where the real value of AI lies. The 56.55 the model scored with the goal known and only the path to predict is above the straight line's 51.89—which is to say that with the goal clear, the model really can plan better than a straight line. This is the conclusion of Section 34.6: do not ask whether a language model can do

navigation, but which parts of navigation suit being left to it.

One last word is worth adding. The human expert scores 75.40 on this benchmark against the best model's 34.38—a gap of 41.02 points. The technology is still moving quickly as this book reaches its last page; but for the foreseeable future, that body of geometric method humanity took forty years to build remains the fundamental guarantee of whether a robot can reach its destination safely.

Implementation: a Campus Semantic Navigation System

This is the final implementation of the book, in four stages, and how far it is carried may be chosen by the level of the course. It draws on Chapters 32, 33 and 34 together.

Stage One A Dataset of Campus Objects (suitable for high school)

1. Define the categories: discuss with the class and settle on five to ten objects common about a campus (suggested: classroom doors, whiteboards, lecterns, chairs, stairs, lifts, corridors, waste bins, hydrants, water coolers, barriers).
2. Plan the viewpoint: decide the height and angle of the shots, and be sure to imitate the robot's real viewpoint (30 to 50 centimeters high) rather than photographing while standing—the most important detail of this implementation.
3. Collect in groups: each group takes two or three categories and photographs at least 200 images of each, covering different angles, distances, lighting and occlusion, and deliberately including difficult samples.
4. Label: set a standard for labeling first (does a half-hidden door count? where does the box go?), then label, and spot-check at least 10 percent of one another's

work.

5. Split the dataset: split by the place of photography rather than at random, to avoid an inflated assessment.
6. Train the model, assess it on the test set, and analyze the errors of the categories that do worst.

Stage Two Semantic Search and the Common-Sense Prior (the core of this chapter)

This stage needs no real robot and can be done on a map alone; it is a reproduction of the three figures of this chapter.

1. Build a semantic map of the floor: digitize the plan of your own departmental building and mark the type of every room and the position of its door.
2. Build the distribution of the objects: survey how often five to eight objects actually appear in each kind of room. This step is a piece of valuable fieldwork in itself.
3. Implement the two orders of search: geometric and semantic, run each several hundred times over randomly placed objects, and compare the distributions of the distance searched. This is your Figure 34-1.
4. Sweep the quality of the prior: shuffle or reverse the prior deliberately, measure how the distance searched changes, and find the crossing point at which the prior becomes worth using. This is your Figure 34-2.
5. Design the way out: implement the strategy of falling back on the geometric order when the first two semantic suggestions fail, and measure how much it recovers when the prior is wrong.

Stage Three A Local Knowledge Base and a Language Interface

1. Build the knowledge base: by the structure of Figure 34-3, write out the topology of the floors of your own building, the attributes of its places, the rules of passage and what common queries correspond to. A plain text file will serve.
2. Implement the retrieval: given one instruction, retrieve the relevant entries from the knowledge base. Keyword matching will do to begin with; there is no need to start with vector retrieval.
3. Attach the language model: have the model output a structured goal from what was retrieved (the name of the region, the coordinate, the confidence, the mark of the source).
4. Test the failing case: ask deliberately for a place not in the knowledge base, and confirm that the system marks the answer a guess rather than pretending to know.

Stage Four The Verification Layer and End-to-End Integration

1. Measure the success of every step: divide your task into n steps, test each separately 30 times and more, and obtain the p_i of each.
2. Predict the overall success: compute the theoretical value by Formula One, then run 30 complete tasks and compare the prediction with the measurement. The difference is itself good material for discussion.
3. Add verification: design a check for every step, and measure the check's detection rate r (create failures deliberately and see how many the check catches).
4. Measure the improvement: measure the overall success again with verification and retrying added, and compare with the prediction of Formula One. This is

your Figure 34-5.

5. Write the report: the heart of the report is not that we built it but what we measured, where it disagreed with the prediction, and why. This is what the book has asked for consistently from Chapter 1 to Chapter 34.

Further Reading

- The benchmark for assessing vision-language models on embodied navigation (2025). (The source of Section 34.6 and Case 34-3, with the complete method of scoring.)
- A survey of vision-language navigation enhanced by large models (2026). (The basis of Section 34.6 on the present state of real deployment.)
- Research on open-vocabulary zero-shot object navigation. (The theoretical foundation of the semantic exploration of Section 34.3.)
- The semantic maps and scene graphs of Chapter 32, and the training in simulation and edge deployment of Chapter 33. (The direct precursors of this chapter; the three chapters make up Part IX.)
- The hybrid architecture of Chapter 3, the degradation design of Chapter 26, and the methods of assessment and debugging of Chapter 31. (The source of the thinking behind the layered architecture and the mechanism of verification of Section 34.5.)
- The probability and uncertainty of Chapter 11. (The theoretical foundation of uncertainty must be passed down in Section 34.2.)

The Points of This Chapter

- The task level differs from the perceptual levels in kind: the input of perception is a sensor and its output a description of the world; the input of the task level is a person's intention and its output an action. An intention is often vague, and the better half of the task level's work is supplying the assumptions that were not spoken.
- Language to action falls into five stages: parsing the intention, locating the place, locating the goal, planning the path, executing and confirming. Everything after the fourth stage the first thirty-one chapters have dealt with entirely; language navigation attaches a stretch of conversion in front of SLAM and does not replace it.
- What an upper layer sends a lower one must be structured data (coordinates, lists, confidences, marks of source) and not free text—free text can neither be parsed nor verified. Having a language model output velocity commands directly turns a system that can be checked into a black box that cannot.
- The common-sense prior is worth a good deal: under the same map, the mean distance searched in the semantic order is 36 meters against the geometric order's 90, a shortening of 60.1 percent (81.9 by the median). But in 15.3 percent of the trials the semantic order went farther—it improves the expectation and not every single run.
- The benefit depends on how concentrated the object's distribution is: the ladder (70 percent in the storeroom) is shortened by 65 percent and the copier (65 percent in an office) by 77, while the fire extinguisher (60 percent in the corridor) is shortened by only 9. For objects that are everywhere, the return on building semantic reasoning is low.
- The accuracy α of the prior and the distance searched are roughly linear (110

meters at α = 0, 35 at α = 1), crossing the geometric order at about α = 0.25. A wholly random prior (110 meters) is already a little worse than using none (92); what is really dangerous is a prior that points the wrong way—146 meters, 60 percent worse than using none at all. And a confident, wrong prior of just this kind is what a language model is most apt to produce.

- RAG supplies with a local knowledge base the institutional knowledge a model lacks, and its three advantages are that it can be maintained, that it can be verified, and that it is cheap. The common-sense prior (statistical, across sites, answering where to look first) and the knowledge base (definite, particular to a site, answering where that place is) do different offices, and a good system wants both.
- The layered architecture divides by time scale: the task layer (seconds), the semantic layer (0.1–1 second), the geometric layer (10–30 Hz) and the control layer (100 Hz and above). The four practical points are asynchronous design, an explicit interface, a mechanism of refusal, and a path of degradation. The principle of safety is that an upper layer may go wrong and a lower layer must hold the line.
- The success of a multi-step task is a product: an eight-step task is only 43.0 percent at 90 percent per step and a mere 16.8 at 80. This explains why every subsystem looks all right and the whole keeps failing.
- Verification and retrying do far more than raising the ability of a single step: an eight-step task at 80 percent per step, with verification at a detection rate of 0.5 (and at most two retries), rises from 16.8 percent to 38.7, and to 93.8 as the detection rate approaches 1—with the underlying ability entirely unchanged. Verification is usually much easier than execution, and that asymmetry is the

fundamental reason a layered architecture can stand.

- The embodied navigation benchmark of late 2025 shows the best vision-language model scoring 34.38 against the human expert's 75.40, a gap of 41.02. More notable still, a straight line to a known goal scores 51.89, beating the model's complete trajectory; while the same model, with the goal known and only the path to predict, reaches 56.55, and predicting the goal point alone drops to 29.65. The model can walk and cannot recognize—locating the goal is the real bottleneck at present.
- The four great risks are hallucination, the systematic character of a misjudgment, latency, and security. Of these the systematic misjudgment is the most dangerous, because it breaks the assumption that retries are independent, and retrying cannot save it.
- The thread running through the book is that only by understanding a limit can a limit be broken. AI lets a robot understand and think things through, while walking steadily and not running into people still rests on the geometry and control of the first thirty-one chapters. For the foreseeable future, neither can be spared.

The Close of the Book

Part IX is complete here, and with it the book.

From Chapter 32, which gave the map a meaning, through Chapter 33, which taught the robot to move, to this chapter, which let it understand human speech, these three chapters have carried the geometric foundation of the thirty-one before to the frontier of application. But remember those numbers of Section 34.6: the most advanced model is still far from human at embodied navigation, and a straight line to a known goal even

beats its complete planning.

This is not to pour cold water but to see the position clearly. The geometric and probabilistic methods you have learned from Chapter 1 to Chapter 31 are not an old technique about to be replaced by AI; they are the ground any language model must stand upon.

Thirty-four chapters, nine parts, and here we are. What to build next is for you to decide.